

Visual analytics system building

CS524: Big Data Visualization & Analytics
Fabio Miranda



urbantk.org



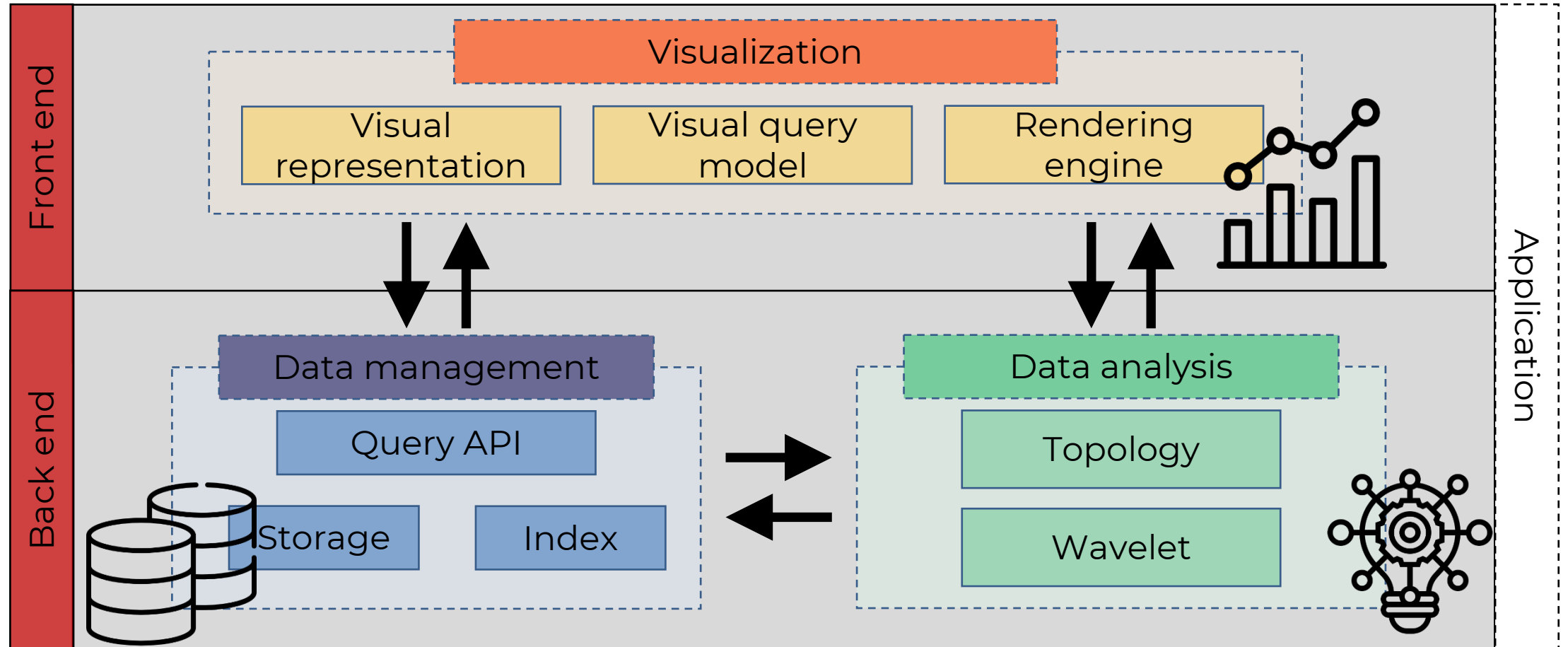
evl

electronic
visualization
laboratory



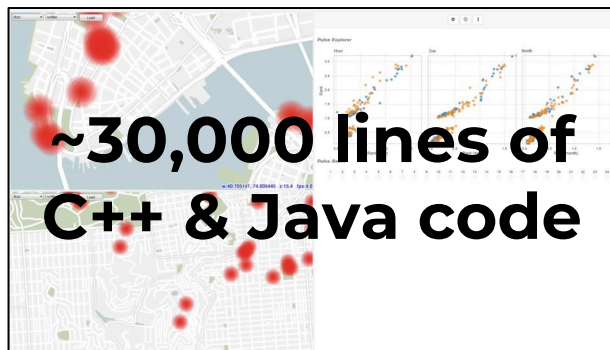
UNIVERSITY OF
ILLINOIS CHICAGO

Visual analytics system

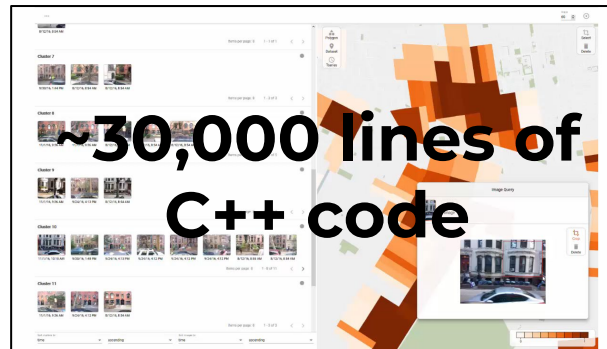


Building VA systems: Understand → Ideate → Make → Evaluate

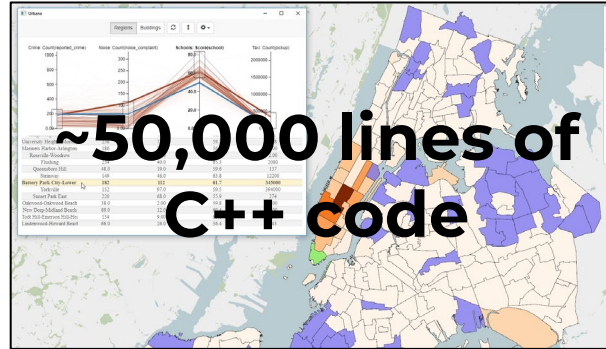
- Real-world problems are becoming more complex, and so are the data and VA systems used to tackle them.
- How are these systems built in practice?



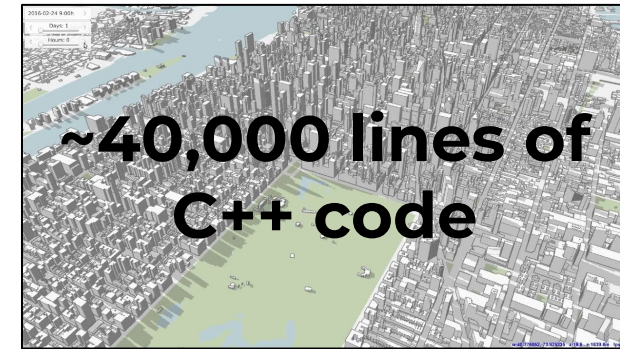
Urban Pulse:
Large-scale data mining of social media data



Urban Mosaic:
Interactive exploration of large imagery data



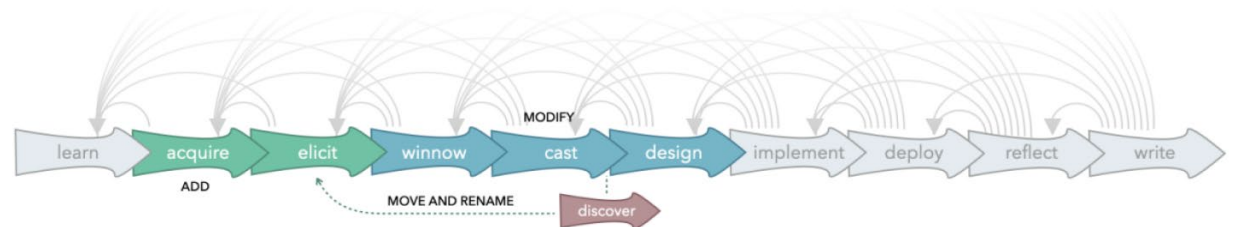
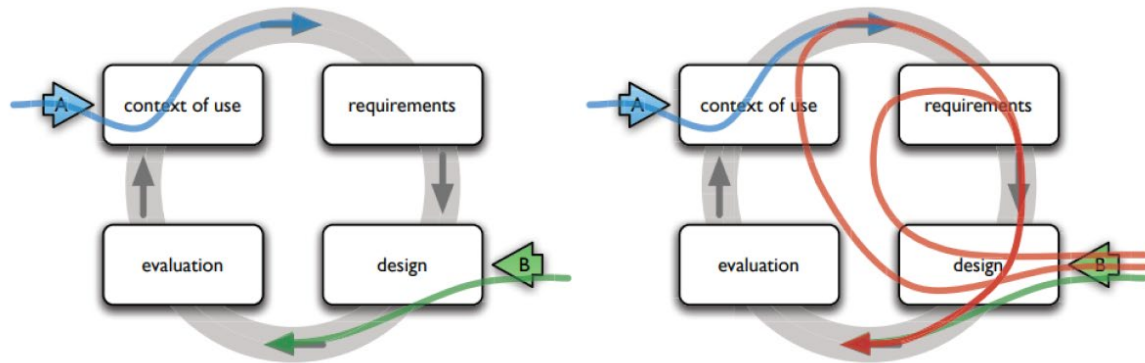
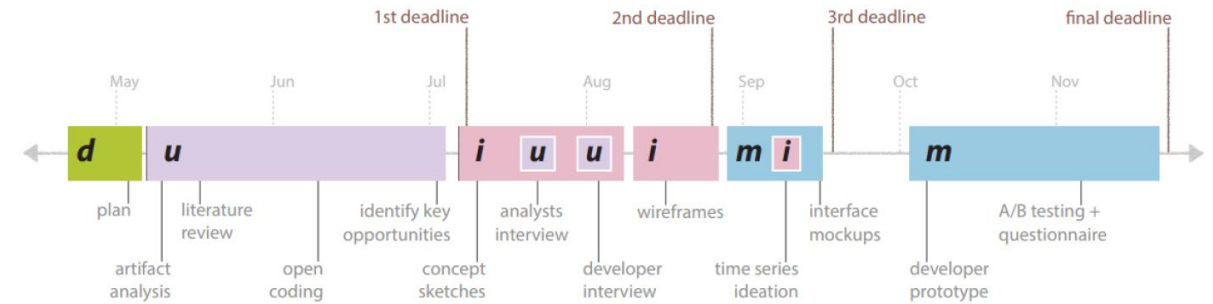
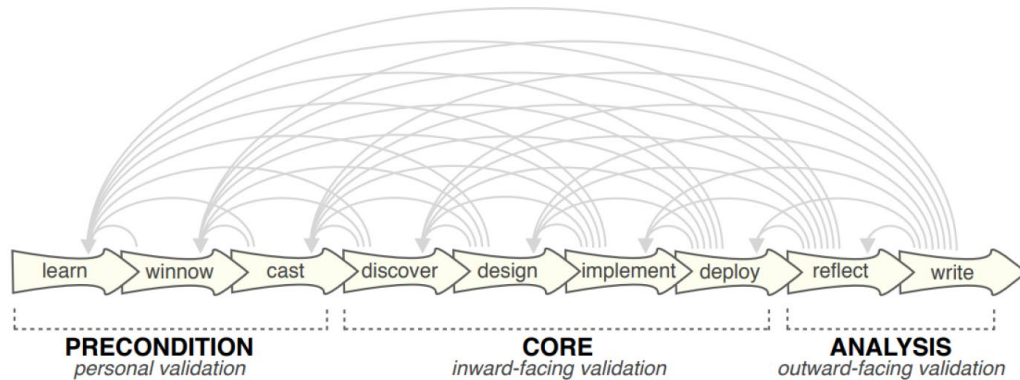
Urbane:
Interactive exploration of large data



Shadow Profiler:
City-scale assessment of sunlight access

Building VA systems: Understand → Ideate → Make → Evaluate

- Real-world problems are becoming more complex, and so are the data and VA systems used to tackle them.
- How are these systems built in practice?

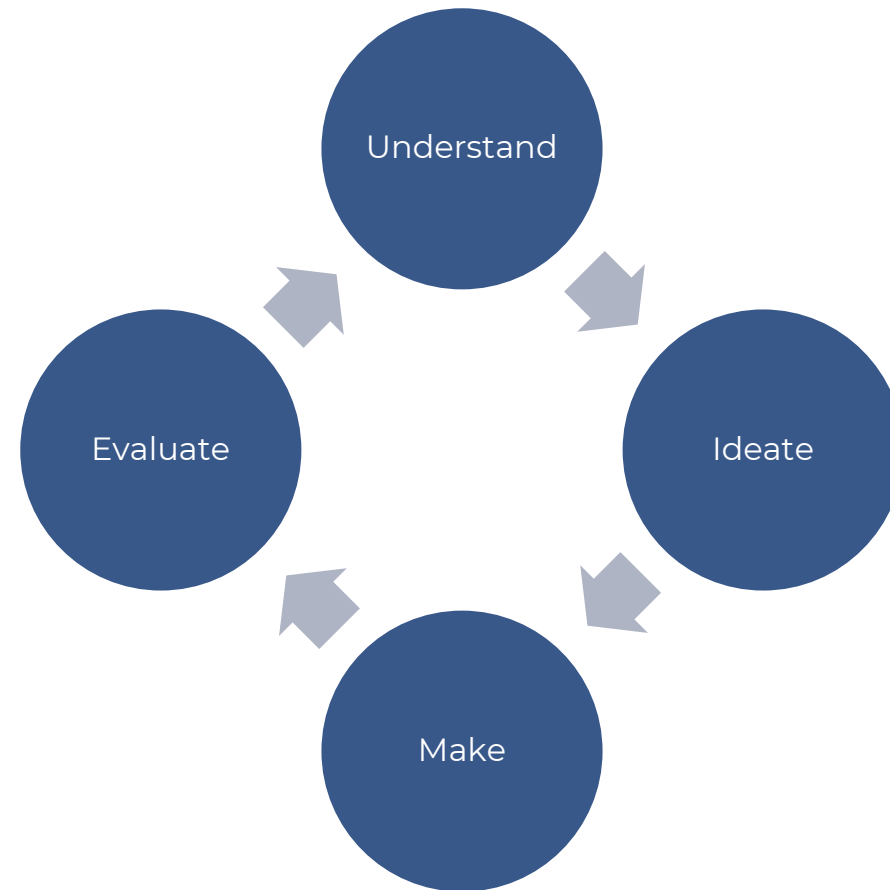


Building VA systems:

Understand → Ideate → Make → Evaluate

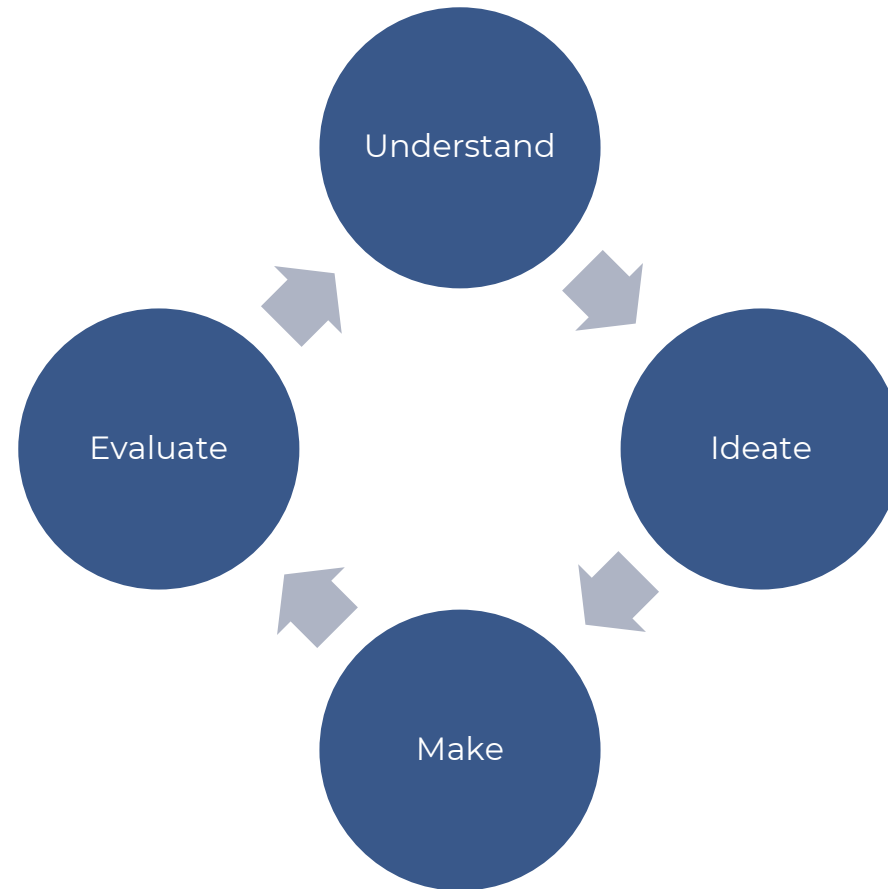
- Real-world problems are becoming more complex, and so are the data and VA systems used to tackle them.
- How are these systems built in practice?
 - Lloyd and Dykes, 2011:
 - Context of use → Requirements → Early prototypes → Later prototypes → Full application → Evaluation
 - Sedlmair et al., 2012:
 - Learn → Winnow → Cast → Discover → Design → Implement & deploy → Reflect & write
 - McKenna et al., 2014:
 - Understand → Ideate → Make → Deploy
 - Hall et al., 2019:
 - Data analysis → Study methods → Prototyping → Learning → Communicating
 - Oppermann and Munzner, 2020:
 - Learn → Acquire data → Elicit tasks → Winnow → Cast → Design → Implement & deploy → Reflect & write
 - Software engineering:
 - Requirements gathering → Design → Development → Testing → Deployment & maintenance

Building VA systems: Understand → Ideate → Make → Evaluate



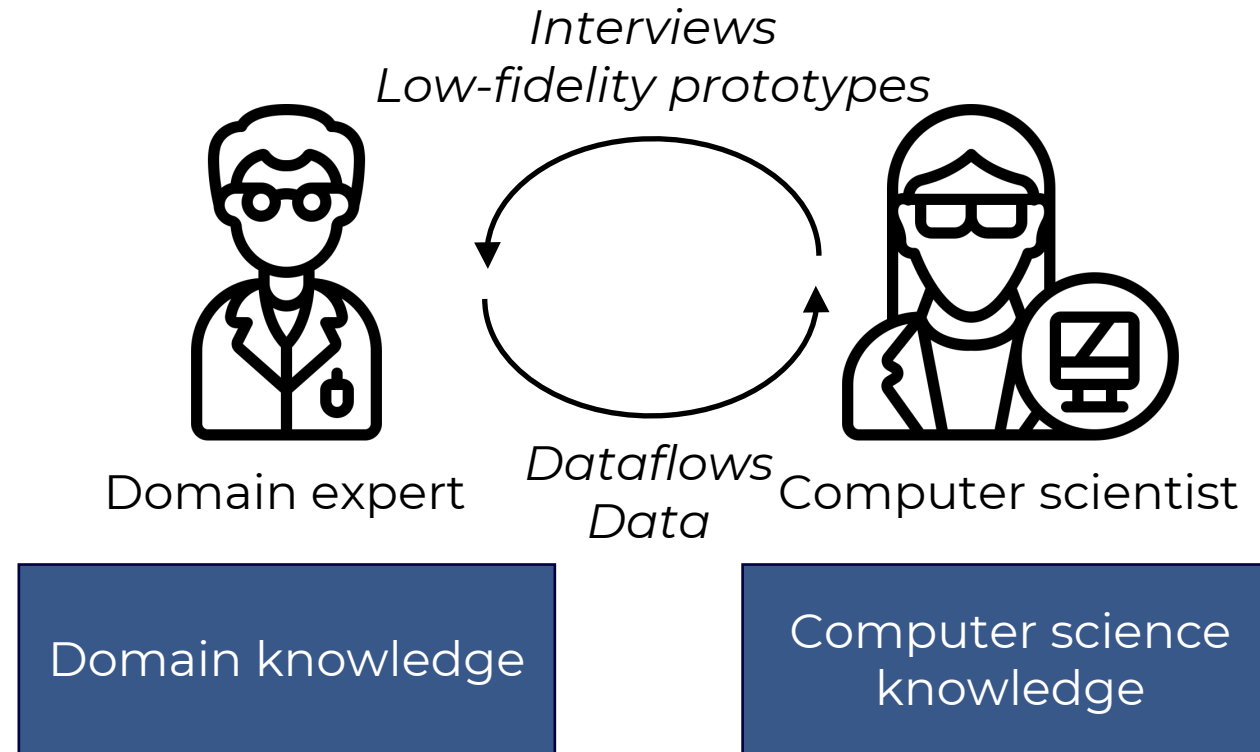
Important: design study methodologies can usually be mapped to this four-stage process.

Building VA systems: Understand → Ideate → Make → Evaluate

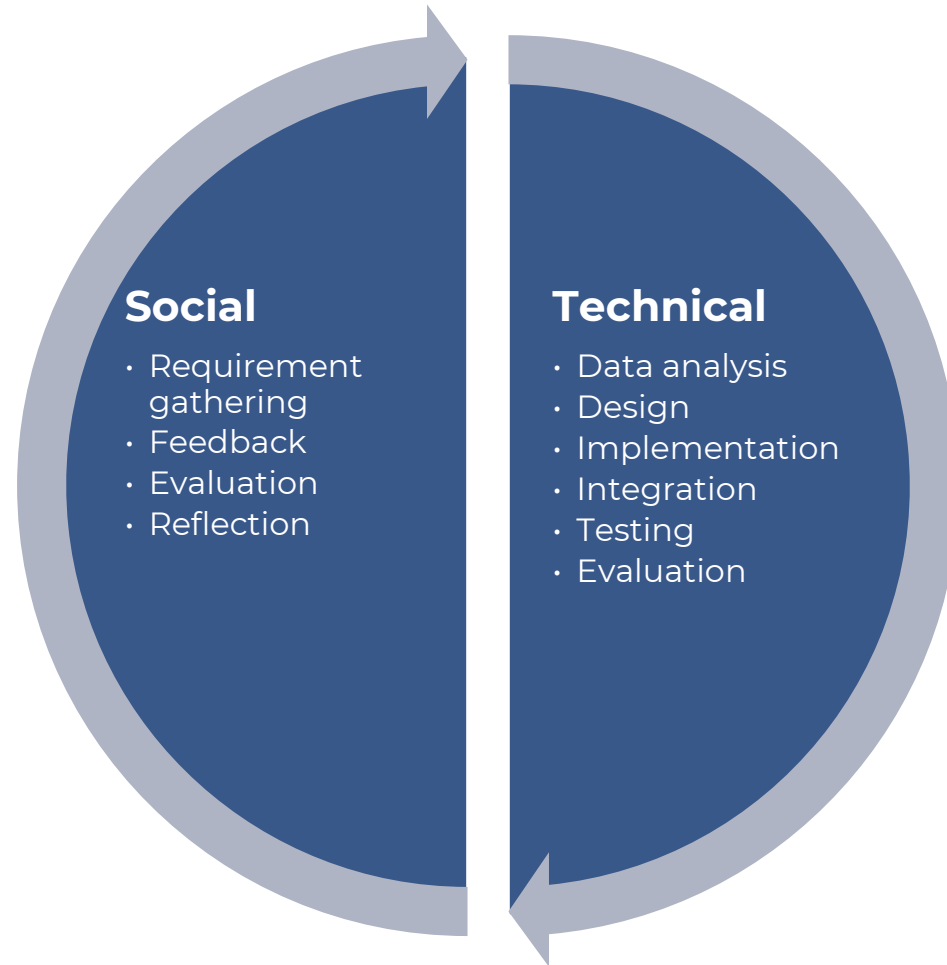


Important: design study methodologies can usually be mapped to this four-stage process.

Building VA systems: Understand → Ideate → Make → Evaluate



Building VA systems: Understand → Ideate → Make → Evaluate



Building VA systems: Understand → Ideate → Make → Evaluate



Building VA systems: Understand → Ideate → Make → Evaluate

- Building VA systems has been limited by the lack of “structured” guidance for how systems are composed and implemented.
- **AI is shifting the bottleneck:**

**From “*Can we build it?*”
To “*Can we explore & justify choices and
iterate faster without losing rigor?*”**



New approaches for VA system building

8796

IEEE TRANSACTIONS ON VISUALIZATION AND COMPUTER GRAPHICS, VOL. 31, NO. 10, OCTOBER 2025

Visualizationary: Automating Design Feedback for Visualization Designers Using Large Language Models

Sungbok Shin[✉], Sanghyun Hong[✉], and Niklas Elmqvist[✉], Fellow, IEEE

Abstract—Interactive visualization editors empower users to author visualizations without writing code, but do not provide guidance on the art and craft of effective visual communication. In this article, we explore the potential of using an off-the-shelf large language models (LLMs) to provide actionable and customized feedback to visualization designers. Our implementation, Visualizationary, demonstrates how ChatGPT can be used for this purpose through two key components: a preamble of visualization design guidelines and a suite of perceptual filters that extract salient metrics from a visualization image. We present findings from a longitudinal user study involving 13 visualization designers—6 novices, 4 intermediates, and 3 experts—who authored a new visualization from scratch over several days. Our results indicate that providing guidance in natural language via an LLM can aid even seasoned designers in refining their visualizations.

Index Terms—Visualization design, design critique, feedback, human-centered AI, large language models.

I. INTRODUCTION

TODAY DEMOCRATIZE the transformative power of data in today's information society, it is not sufficient that people get access to visualizations created by others; they must also be empowered to author their own visualizations. In response, recent years have seen an influx of interactive tools that enable users to create visualizations without writing code, such as iVis-Designer [51], Data Illustrator [38], and Lyra [56]. However, no matter how advanced these tools become, a fundamental barrier remains: authoring effective visualizations is a complex task

Received 23 July 2024; revised 31 May 2025; accepted 11 June 2025. Date of publication 13 June 2025; date of current version 5 September 2025. This work was supported in part by the U.S. National Science Foundation (Shin) under Grant IIS-1901485, in part by Google Faculty Research Award (Hong), and in part by Villum Investigator under Grant VL-54492 by Villum Fonden (Elmqvist). Recommended for acceptance by A. Endert. (Corresponding author: Niklas Elmqvist.)

This work involved human subjects or animals in its research. Approval of all ethical and experimental procedures and protocols was granted by University of Maryland College Park (UMCP), College Park, MD Institutional Review Board under Application No. 1185917-8 and Exemption category #45CR46.104(4)(2)(1-4).

Sungbok Shin is with Aviz at Inria, Université Paris-Saclay, 91190 Saclay, France (e-mail: sungbok.shin@inria.fr).

Sanghyun Hong is with the Department of Computer Science, Oregon State University, Corvallis, OR 97331-4501 USA (e-mail: sanghyun.hong@oregonstate.edu).

Niklas Elmqvist is with the Department of Computer Science, Aarhus University, 8200 Aarhus, Denmark (e-mail: elm@cs.au.dk).

All our supplemental materials are available at <https://osf.io/v7hu8>.

Digital Object Identifier 10.1109/TVCG.2025.3579700

1077-2626 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on February 16, 2026 at 20:44:40 UTC from IEEE Xplore. Restrictions apply.

requiring visual design skills, an understanding of aesthetics, and experience—expertise that not all would-be designers possess.

Fortunately, visualization designers have nothing to lose but their chains. We introduce Visualizationary, a system to seize the means of visualization design by using off-the-shelf large-language models (LLMs). Our system enhances the design of communicative visualizations by providing perceptual feedback. While it may not fully replace feedback from peers or colleagues, it serves as a valuable alternative when it is impossible to gain human feedback and can benefit designers of all expertise levels, from novices to seasoned ones.

Although competing approaches for improving visualization designs, such as heuristics (like small, smart defaults in Lyra) and Grammar of Graphics (GoG)-style specifications have their own advantages, they also carry limitations due to their deterministic nature. Heuristic-driven methods lack human intervention, which can constrain creativity and limit the potential for novel ideas. GoG-style specifications, despite offering precise chart descriptions, lead to producing standardized solutions, reducing the diversity of possible visualization outcomes.

In contrast, Visualizationary automates perceptual feedback using a suite of automated techniques that simulate how a viewer would perceive a visualization. By providing actionable insights without the need to standardize changes, Visualizationary preserves the designer's creativity while enhancing effectiveness.

The basic idea behind Visualizationary is to investigate how an LLM can be used to aid the iterative design process of a visualization artifact in a workflow that we call *analyze-clarify-guide-track* (ACGT):

- 1) **Analyze** the state of a visualization using automated perceptual filters, yielding a corresponding analysis report;
- 2) **Clarify** the results from the filters in the report such that it is understandable to even novice visualization designers;
- 3) **Guide** the designer in making changes that address identified concerns in their visualization; and
- 4) **Track** the trajectory of the visualization artifact over time during its iterative design process.

We present a web-based implementation of Visualizationary that leverages a server-side LLM, demonstrating how a “vanilla” LLM can be used for this highly complex visualization design task. Visualizationary operates as follows: We first translate chart images into text using a vision-language model (VLM) [36] and then generate automatic design guidance using an LLM as a template and text processor. Users can also upload their existing

IEEE TRANSACTIONS ON VISUALIZATION AND COMPUTER GRAPHICS, VOL. 31, NO. 10, OCTOBER 2025

7089

From Requirement to Solution: Unveiling Problem-Driven Design Patterns in Visual Analytics

Yuchen Wu[✉], Shengnan Gao[✉], Shizhen Zhang[✉], Xiaofeng Dou[✉], Xingbo Wang[✉], and Quan Li[✉]

Abstract—Visual Analytics (VA) researchers frequently collaborate closely with domain experts to derive requirements and select appropriate solutions to fulfill these requirements. Despite strides made in exploring requirement and solution spaces, challenges persist due to the absence of guidance in the initial consideration space and the lack of shared problem-solving knowledge, often resulting in suboptimal solutions. To address these issues, we conducted an empirical study of VA research, with a focus on mapping the relations between requirement and solution spaces. Analyzing 220 VA papers, we formulate refined topologies for data, requirements, and solutions. We propose conceptualizing the connections between requirements, data, and solutions through knowledge graphs and utilizing solution paths to encapsulate fundamental problem-solving knowledge in visual analytics research. Through the integration of solution paths into a graph and analyzing their interconnections, we identified a subset of problem-driven design patterns that demonstrated the efficacy of our approach. By externalizing problem-solving knowledge and formulating problem-driven design patterns, our aim is to streamline the exploration of consideration space, facilitating the inclusion of “good” solutions, and establish a benchmark for shared design decisions among researchers and readers.

Index Terms—Visual analytics, design patterns, problem-solving, typology.

I. INTRODUCTION

VISUAL Analytics (VA) integrates automated analysis techniques with interactive visualizations for effective understanding, reasoning, and decision making based on data [1]. Being an application-oriented discipline, we often derive practical requirements through close collaboration with domain experts. Subsequently, we generate multiple solutions and select a *good* one to meet the requirements [2]. Sedlmair et al. [2] have indicated a common pitfall in this process: considering too

Received 27 July 2024; revised 22 January 2025; accepted 1 February 2025. Date of publication 5 February 2025; date of current version 5 September 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62372296, in part by the Shanghai Engineering Research Center of Intelligent Vision and Imaging, in part by the Shanghai Frontiers Science Center of Human-centered Artificial Intelligence (ShangHAI), and in part by the MoE Key Laboratory of Intelligent Perception and Human-Machine Collaboration (KLIP-HuMaCo). Recommended for acceptance by M. Sedlmair. (Corresponding author: Quan Li.)

Yuchen Wu, Shengnan Gao, Shizhen Zhang, Xiaofeng Dou, and Quan Li are with the School of Information Science and Technology, Shanghai Engineering Research Center of Intelligent Vision and Imaging, ShanghaiTech University, Shanghai 201210, China (e-mail: wuyuchen36@shanghaitech.edu.cn; gaoshn1@shanghaitech.edu.cn; zhangshizhen@shanghaitech.edu.cn; douxf2023@shanghaitech.edu.cn; liquan@shanghaitech.edu.cn).

Xingbo Wang is with Weill Cornell Medical College, Cornell University, Ithaca, NY 14850 USA (e-mail: xingbo.wang@cornell.edu).

Digital Object Identifier 10.1109/TVCG.2025.3538768

1077-2626 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on February 16, 2026 at 20:44:46 UTC from IEEE Xplore. Restrictions apply.

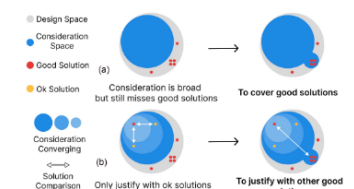


Fig. 1. An ill-positioned consideration space (e.g., with only *bad* and *ok* solutions) results in (a) the selection of *ok* solutions rather than *good* ones, and (b) the justification of current solution with *ok* solutions rather than *good* ones.

few solutions and prematurely committing to a selected one. However, there exists a significant gap between requirements and solutions, leading to a lack of guidance on which options to consider. This gap can result in getting stuck in a “local optimum”, as similarly indicated in previous literature [3], [4]. To systematically develop and justify our VA systems, it is crucial to investigate and elaborate on the mapping between requirements and solution spaces.

Prior research has made strides in refining both requirement and solution spaces of VA systems through investigations into analytical tasks [5], composite visualizations [6], interaction techniques [7], as well as task-specific [8] and data-specific [9] visualizations. However, challenges persist when transitioning from requirements to solutions, emanating from two critical perspectives: 1) *Broad consideration can still miss good solutions.* Methodologically, visualization models [2], [10], [11] have been proposed to shape the structure of VA research. These models generally advocate starting with a broad consideration space and progressively narrowing down to a *good* solution guided by design principles, guidelines, and domain expert discussions. However, these models often prioritize variance (i.e., emphasize “bread” over balancing accuracy. The transition from requirements analysis to design specification often relies on intuitions, and design decisions are frequently undocumented [4]. Therefore, even skilled researchers can miss *good* solutions [12] when the initial consideration space lacks guidance [13], [14], even if the consideration space shifts and expands subsequently. As depicted in Fig. 1(a), an inadequately situated consideration space (e.g., containing only *bad* solutions and *ok* solutions) leads



urbantk.org



UNIVERSITY OF
ILLINOIS CHICAGO

New approaches for VA system building

VA-Blueprint: Uncovering Building Blocks for Visual Analytics System Design

Leonardo Ferreira, Gustavo Moreira, and Fabio Miranda

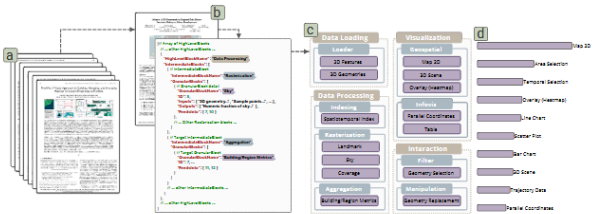


Fig. 1: VA-Blueprint provides a structured way to uncover and organize the fundamental building blocks used in visual analytics (VA) systems. (a) The process begins with the curation of a corpus of VA system research papers. (b) From each paper, core components and their relationships are systematically extracted and structured into a formal, hierarchical blueprint (represented as JSON), detailing the system's components and their dependencies. (c) This blueprint categorizes components across multiple levels of abstraction: high-level stages (e.g., *Data Processing*), intermediate groups (e.g., *Visualization*), and specific granular blocks (e.g., *Scatter Plot*). (d) The final knowledge base comprises 101 machine-readable blueprints encoding component hierarchies, and data and interaction dependencies.

Abstract—Designing and building visual analytics (VA) systems is a complex, iterative process that requires the seamless integration of data processing, analytics capabilities, and visualization techniques. While prior research has extensively examined the social and collaborative aspects of VA system authoring, the practical challenges of developing these systems remain underexplored. As a result, despite the growing number of VA systems, there are only a few structured knowledge bases to guide their design and development. To tackle this gap, we propose VA-Blueprint, a methodology and knowledge base that systematically reviews and categorizes the fundamental building blocks of urban VA systems, a domain particularly rich and representative due to its intricate data and unique problem sets. Applying this methodology to an initial set of 20 systems, we identify and organize their core components into a multi-level structure, forming an initial knowledge base with a structured blueprint for VA system development. To scale this effort, we leverage a large language model to automate the extraction of these components for other 81 papers (completing a corpus of 101 papers), assessing its effectiveness in scaling knowledge base construction. We evaluate our method through interviews with experts and a quantitative analysis of annotation metrics. Our contributions provide a deeper understanding of VA systems' composition and establish a practical foundation to support more structured, reproducible, and efficient system development. VA-Blueprint is available at urbantk.org/va-blueprint.

Index Terms—Visual analytics, large language models, knowledge base, system development, urban visual analytics

1 INTRODUCTION

Visual analytics (VA) systems help users make sense of complex data by combining analytics with interactive visualizations. Their usefulness has been demonstrated across diverse domains, including biology [27], healthcare [28], urban planning [17], and climate science [20]. However, building VA systems remains a highly complex and iterative process. System builders must integrate data processing techniques, analytics capabilities, and visualization strategies while ensuring interpretability for domain experts and maintaining efficiency to support interactivity. Despite their significance and inherent challenges, VA

• Leonardo Ferreira, Gustavo Moreira, and Fabio Miranda are with the University of Illinois Chicago. E-mail: {lferri10, gmorei3, fabiojm}@uic.edu.

Received 31 March 2025; revised 1 July 2025; accepted 15 July 2025.
Date of publication 4 December 2025; date of current version 3 February 2026.
Digital Object Identifier no. 10.1109/TVCG.2025.3634809

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on February 16, 2026 at 22:46:13 UTC from IEEE Xplore. Restrictions apply.
Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications_standards/publications_standards_info for more information.

system development is often approached in an ad-hoc manner, with minimal reuse of existing components. Given the necessity to address the unique analytical and visualization needs of specific domains, these systems are usually bespoke solutions, leading to fragmented development efforts, making them difficult to extend, adapt, or reuse across projects. This tension, between the need for tailored solutions and the demand for scalable, extensible systems, presents a fundamental challenge in VA system development [12, 58]. On one hand, VA systems must be grounded by domain-specific data, tasks, and analytical workflows: a visualization technique that works well for transportation may be inadequate for urban planning, just as an analytical model for climate science may not be applicable to public health. On the other hand, as VA systems become increasingly complex, the lack of systematic and reusable components hampers the design process and increases the effort needed for development.

Currently, there is a lack of practical knowledge bases that document the building blocks of VA systems. Initial steps have produced valuable taxonomies for visualization tasks and techniques [5]. More recently, new initiatives have emerged to catalog and structure design elements

A Multimodal Framework for Understanding Collaborative Design Processes

Maurice Koch, Nelusa Pathmanathan, Daniel Weiskopf, and Kuno Kurzahls

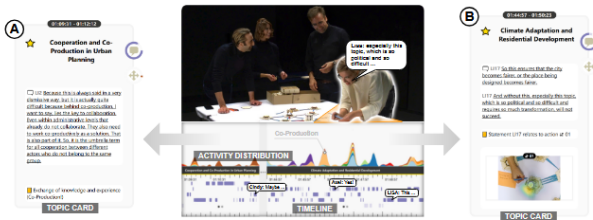


Fig. 1: We record multimodal data from collaborative workshops to derive artifacts from discussions. Streamgraphs show activity and visual attention to different regions of the working area. Activity timelines indicate when different roles in the workshop contributed to a discussion. Editable topic cards are derived automatically from transcripts, serving as elements to build topic summaries. The examples show four participants who discuss the topic of Co-Production in urban planning. Lisa is currently active in the part of the working area that concerns Co-Production (a). Topic Cards (A) and (B) provide a multimodal summary of two key events concerning this discussion that the analyst has authored. (Images edited with Stable Diffusion for privacy).

Abstract—An essential task in analyzing collaborative design processes, such as those that are part of workshops in design studies, is identifying design outcomes and understanding how the collaboration between participants formed the results and led to decision-making. However, findings are typically restricted to a consolidated textual form based on notes from interviews or observations. A challenge arises from integrating different sources of observations, leading to large amounts and heterogeneity of collected data. To address this challenge, we propose a practical, modular, and adaptable framework of workshop setup, multimodal data acquisition, AI-based artifact extraction, and visual analysis. Our interactive visual analysis system, reCAPi, allows the flexible combination of different modalities, including video, audio, notes, or gaze, to analyze and communicate important workshop findings. A multimodal streamgraph displays activity and attention in the working area, temporally aligned topic cards summarize participants' discussions, and drill-down techniques allow inspecting raw data of included sources. As part of our research, we conducted six workshops across different themes ranging from social science research on urban planning to a design study on band-practice visualization. The latter two are examined in detail and described as case studies. Further, we present considerations for planning workshops and challenges that we derive from our own experience and the interviews we conducted with workshop experts. Our research extends existing methodology of collaborative design workshops by promoting data-rich acquisition of multimodal observations, combined AI-based extraction and interactive visual analysis, and transparent dissemination of results.

Index Terms—Collaborative workshops, multimodal analysis of design processes, combination of AI and interactive visualization, design study methodology, eye tracking, reCAPi

1 INTRODUCTION

Design processes in the field of visualization, as well as other domains, are often part of a collaborative effort, comprising multiple stages from acquiring knowledge about potential means to solve a problem to the implementation and analysis of a new design [45]. One way to address the earlier stages of the design process is conducting creative visualization-opportunities (CVO) workshops, which typically involve domain and visualization expertise to address the problem at hand [25].

• All authors are with Visualization Research Center (VISUS), University of Stuttgart. E-mail: {first name} {last name}@visus.uni-stuttgart.de.

Received 31 March 2025; revised 1 July 2025; accepted 15 July 2025.
Date of publication 16 January 2026; date of current version 3 February 2026.
Digital Object Identifier no. 10.1109/TVCG.2025.3634232

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on February 16, 2026 at 20:44:47 UTC from IEEE Xplore. Restrictions apply.
Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications_standards/publications_standards_info for more information.

During such workshops, participants create fragments (e.g., physical objects, utterances, sketches) as outcomes that foster discussion and potentially support decision-making. These fragments further serve as a means to document the whole process by textual summarization of findings, images, or objects. If researchers want to communicate how these results were derived, a temporal analysis of the workshop is required, by someone taking notes throughout or by recording audio and/or video, leading to an increased analysis effort.

Workshop analysis often relies on open coding [12] of recorded audio and handwritten notes, which is time-consuming and requires extensive analysis of the material. The high workload associated with established workflows discourages workshop conductors from recording or harnessing material from other sources, such as video or physiological sensors. However, video has the potential of capturing activities that are not accompanied by verbal utterances, such as gestures, activities, and movement. Many aspects of the video can be analyzed automatically

Qualitative Study for LLM-assisted Design Study Process: Strategies, Challenges, and Roles

Shaolin Ruan, Rui Sheng, Xiaolin Wen, Jiachen Wang, Tianyi Zhang, Yong Wang, Tim Dwyer, and Jianan Li

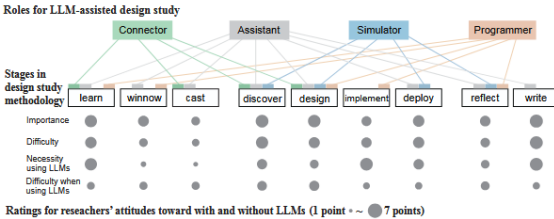


Fig. 1: An overview of the summarized roles of LLMs in the LLM-assisted design study process. Following the framework proposed by Sedlmair et al. [34], we coded all nine stages of the design study by the compiled roles: 1) Connector: LLMs are used to bridge gaps between visualization researchers and domain experts; 2) Assistant: LLMs are commonly used for repetitive tasks to enhance productivity; 3) Simulator: LLMs support simulating user behavior, producing feedback, and testing workflows to uncover usability issues and hidden use cases; 4) Programmer: LLMs are asked to assist in coding tasks, from prototyping to debugging. The bottom section shows ratings from 30 researchers on the importance, difficulty, and necessity of using LLMs, as well as the difficulty when using LLMs at each stage.

Abstract—Design studies aim to develop visualization solutions for real-world problems across various application domains. Recently, the emergence of large language models (LLMs) has introduced new opportunities to enhance the design study process, providing capabilities such as creative problem-solving, data handling, and insightful analysis. However, despite their growing popularity, there remains a lack of systematic understanding of how LLMs can effectively assist researchers in visualization-specific design studies. In this paper, we conducted a multi-stage qualitative study to fill this gap, which involved 30 design study researchers from diverse backgrounds and expertise levels. Through in-depth interviews and carefully-designed questionnaires, we investigated strategies for utilizing LLMs, the challenges encountered, and the practices used to overcome them. We further compiled the roles that LLMs can play across different stages of the design study process. Our findings highlight practical implications to inform visualization practitioners, and also provide a framework for leveraging LLMs to facilitate the design study process in visualization research.

Index Terms—Design Study, Large Language Models (LLMs), Qualitative Study, Visualization

1 INTRODUCTION

Sedlmair et al. introduced the *term design study* to describe an applied research methodology that focuses on creating visualizations to address specific, real-world problems [34]. More specifically, they defined a visualization design study as "a project in which visualization researchers analyze a specific problem faced by domain experts and design a visualization system that supports solving this problem" [34]. Building upon this definition, a nine-stage methodology framework was proposed for conducting design studies, as shown in Fig. 1, which has become a common guidance for this field of research in the visualization community. Design studies are crucial because they bridge the gap between theoretical visualization research and practical applications, ensuring that visualization tools are both usable and useful in real-world contexts [29]. Basically, researchers conduct design studies through labor-intensive processes (e.g., connecting with domain experts, extracting requirements via interviews, and iteratively designing and developing visualization systems), which require significant effort and refined knowledge to gain effective solutions.

Meanwhile, Large Language Models (LLMs) have shown increasing

• S. Ruan, T. Zhang, and J. Li are with Singapore Management University. E-mail: {shuan, tianyi, jianan}@smu.edu.sg and jianan@smu.edu.sg. S. Ruan is also with Monash University.
• R. Sheng is with the Hong Kong University of Science and Technology. E-mail: rshenga@connect.hk.
• X. Wen and Y. Wang are with Nanyang Technological University. E-mail: xiaolin004@e.ntu.edu.sg and yong-wang@ntu.edu.sg.
• J. Wang is with Zhejiang University. E-mail: wangjiachen@zju.edu.cn.
• T. Dwyer is with Monash University. E-mail: tim.dwyer@monash.edu.
• J. Li and Y. Wang are the co-corresponding authors.

Received 31 March 2025; revised 1 July 2025; accepted 15 July 2025.
Date of publication 10 December 2025; date of current version 3 February 2026.
This article has supplementary downloadable material available at <https://doi.org/10.1109/TVCG.2025.3634820>, provided by the authors.
Digital Object Identifier no. 10.1109/TVCG.2025.3634820

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on February 16, 2026 at 21:03:30 UTC from IEEE Xplore. Restrictions apply.
Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications_standards/publications_standards_info for more information.

New approaches for VA system building

Understand

- Learn about the target domain and the practices, needs, problems, and requirements of the domain expert. [Sedlmair et al., 2012]
- Understand the problem, domain, and target users. Gather information to find user needs. [McKenna et al., 2014]
- Undertake domain-specific data analysis, analyze data collaboratively with domain experts. [Hell et al., 2019]
- Elicitation of tasks from stakeholders. [Oppermann and Munzner, 2020]

Ideate

- Early designs to illustrate the essence of an idea. [Lloyd and Dykes, 2011]
- Digital interactive prototypes. [Lloyd and Dykes, 2011]
- Generation and validation of data abstractions, visual encodings, and interaction mechanisms. [Sedlmair et al., 2012]

Make

- Implementation of the tool. [Lloyd and Dykes, 2011]
- Implementation of software prototypes and tools. [Sedlmair et al., 2012]
- Concretize ideas into tangible prototypes. [McKenna et al., 2014]
- Implementation of the software and tools. [Oppermann and Munzner, 2020]

Evaluate

- See previous weeks!

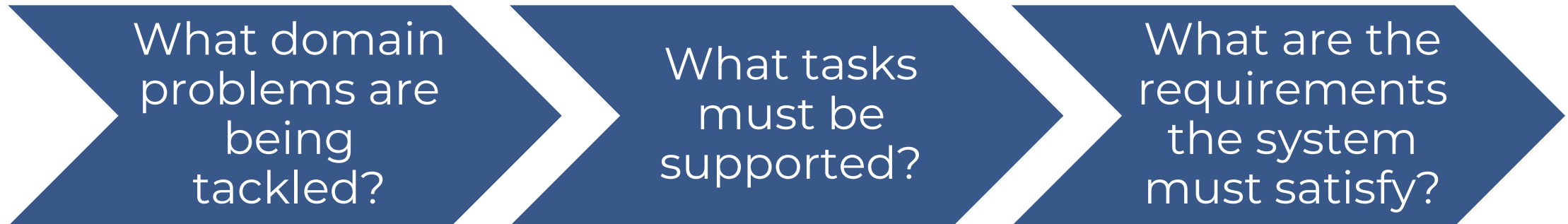
Understand: Problem → Tasks → Requirements

- Goal: Build a shared understand of the domain + users + data + tasks + constraints, and decide what “success” of a VA system means.
- Pain points:
 - Labor intensive, with interviews, observations, workshops, but it's where most of the downstream success is determined.



Understand: Problem → Tasks → Requirements

- Goal: Build a shared understand of the domain + users + data + tasks + constraints, and decide what “success” of a VA system means.
- Pain points:
 - Labor intensive, with interviews, observations, workshops, but it's where most of the downstream success is determined.



Understand: Problem → Tasks → Requirements

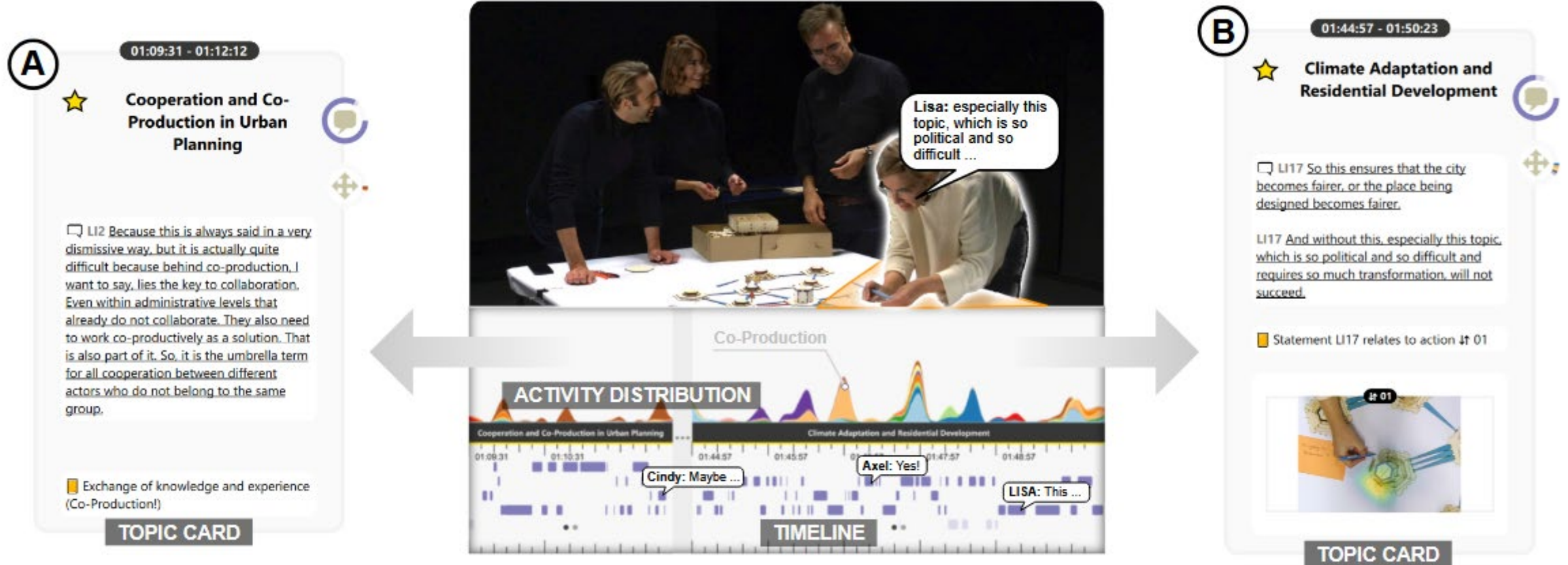
- **Problem framing:** What domain problems are being tackled?
 - A VA system design starts by grounding the work in a real problem faced by domain experts.
 - What is in scope / out of scope?
 - Why this step is hard in VA system building:
 - VA systems are inherently complex + iterative to build and must combine several components while staying interactive.
- **AI opportunities:**
 - **Faster domain ramp-up:**

LLMs help fill “gaps in domain-specific terminology and foundational concepts” (Ruan et al., 2026)
 - **Faster domain translation:**

LLMs can “assist in translating domain-specific knowledge into actionable design insights, decomposing requirements into sub-tasks, and providing design inspiration” (Ruan et al., 2026)

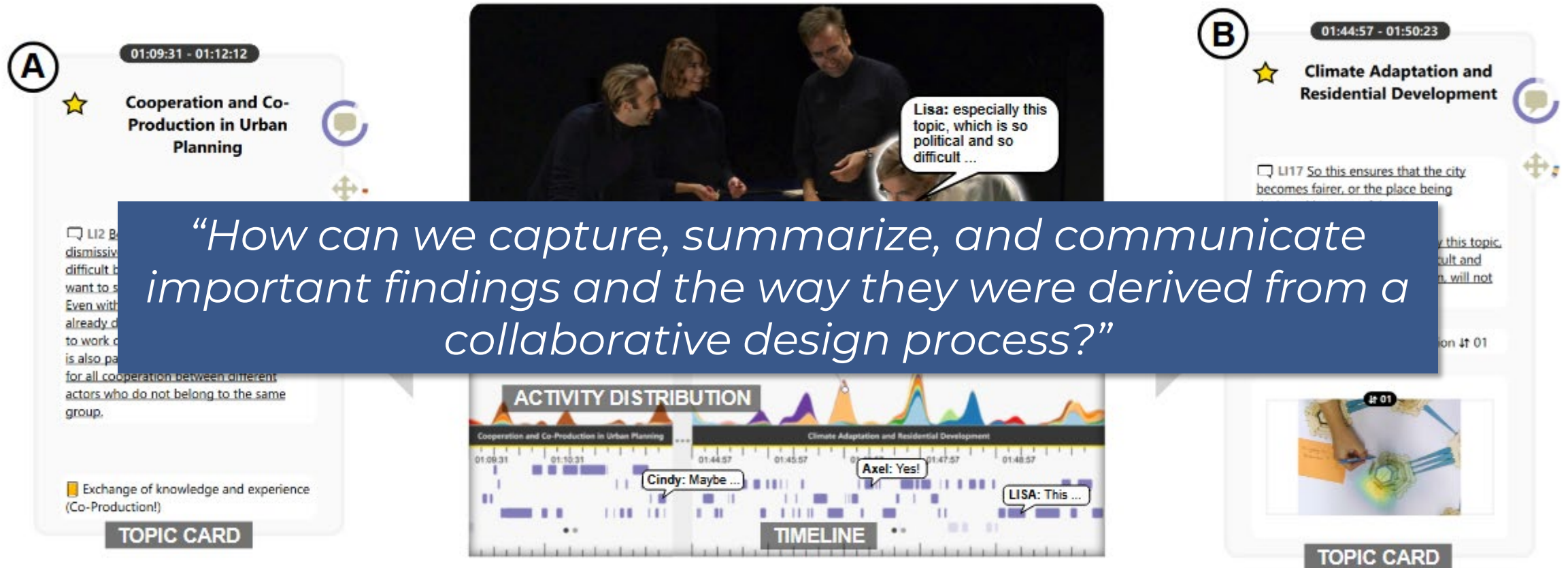


Understand: Problem → Tasks → Requirements



Koch et al., 2026

Understand: Problem → Tasks → Requirements



Koch et al., 2026

Understand: Problem → Tasks → Requirements

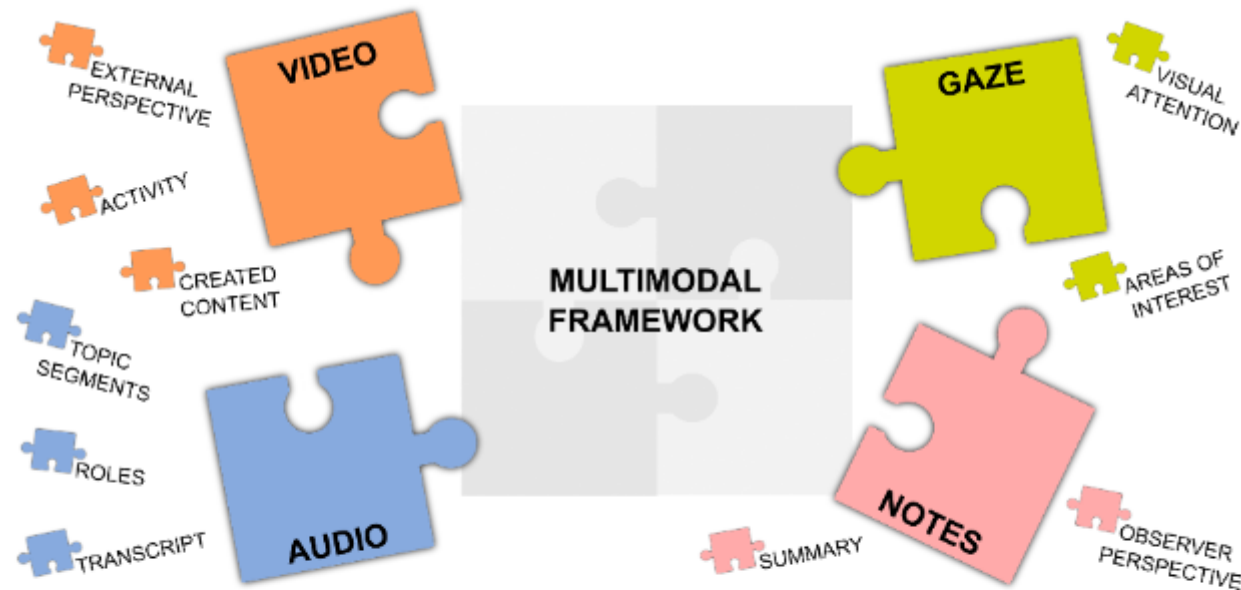
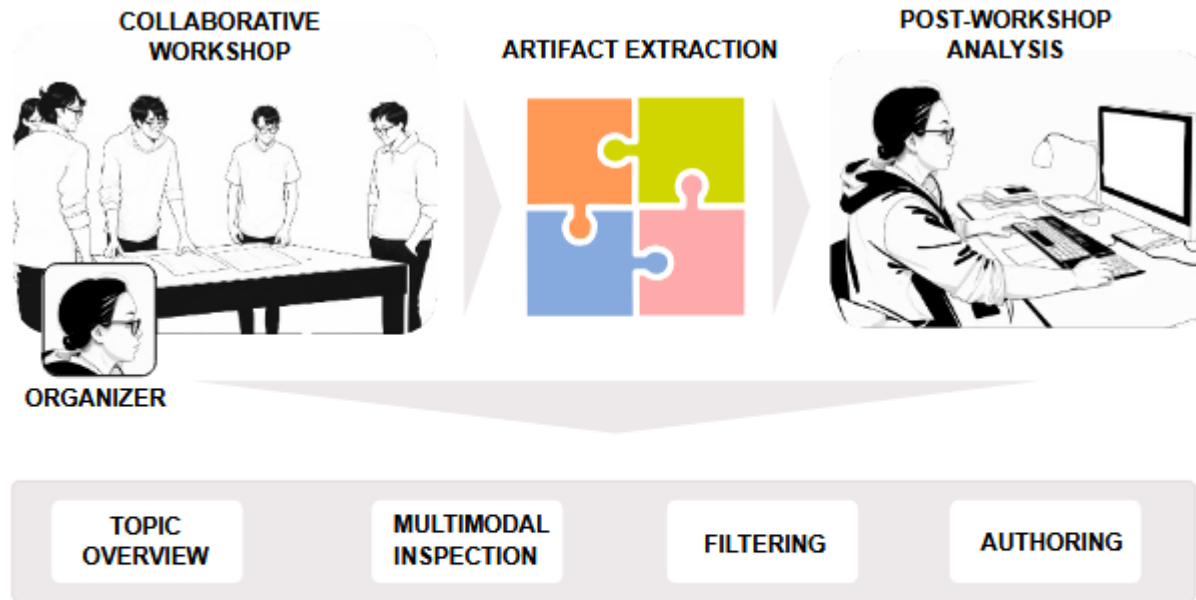


Fig. 2: Our framework supports the flexible combination of recording audio, video, notes, and/or gaze for a multimodal view on collaborative workshop processes. From this data, we extract artifacts for analysis.

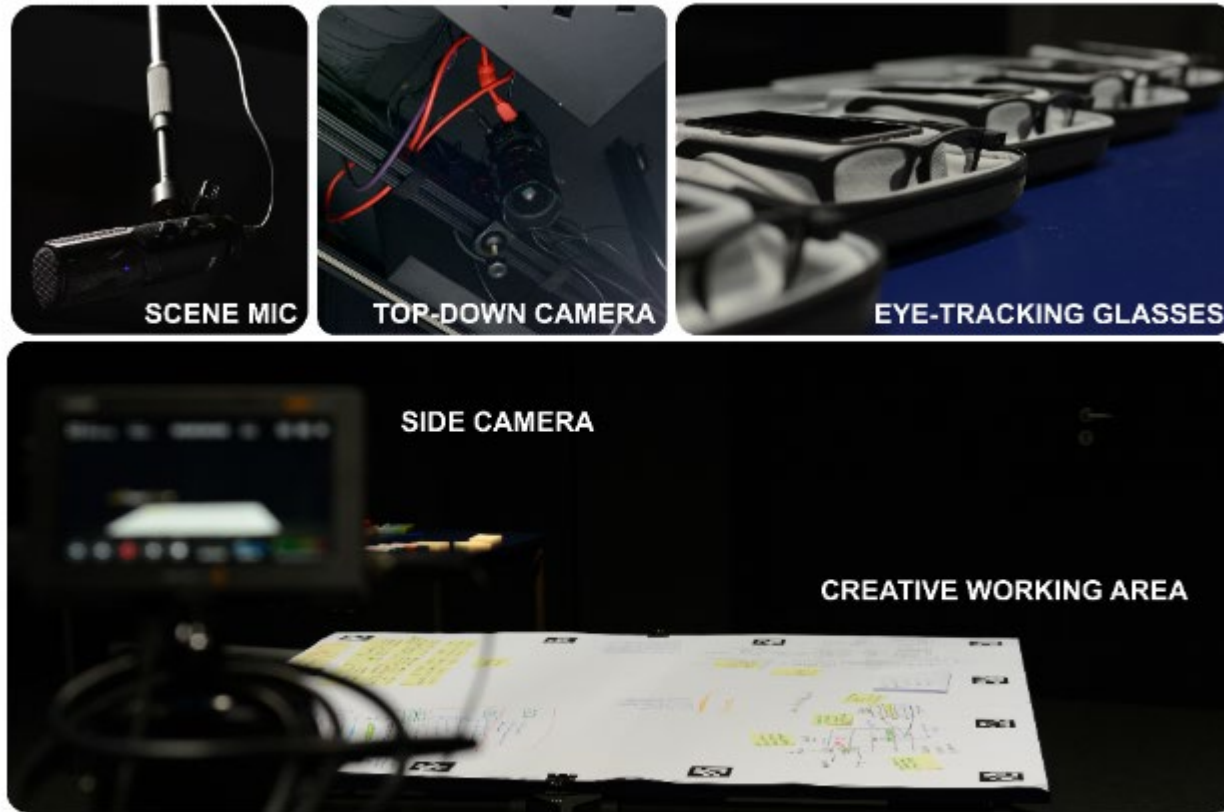
Understand: Problem → Tasks → Requirements



- Multimodal framework for workshop conductors to collect, consolidate, and analyze resulting multimodal data.
- Researchers can revisit important events from different perspectives of the multimodal data.
- Overview of all data streams, and drill-down into each individual stream.

Fig. 4: Data is captured during a workshop and processed to extract artifacts. A post-experiment analysis is conducted by workshop organizers. The analysis comprises multiple methods to inspect and author artifacts to derive summaries for sensemaking and dissemination. *(Images edited with Stable Diffusion for privacy.)*

Understand: Problem → Tasks → Requirements



- Multimodal framework for workshop conductors to collect, consolidate, and analyze resulting multimodal data.
- Researchers can revisit important events from different perspectives of the multimodal data.
- Overview of all data streams, and drill-down into each individual stream.

Fig. 3: Workshop setup with one side camera and one top-down camera. Audio was recorded with a scene microphone mounted above the work area, gaze with multiple eye-tracking glasses.

Understand: Problem → Tasks → Requirements

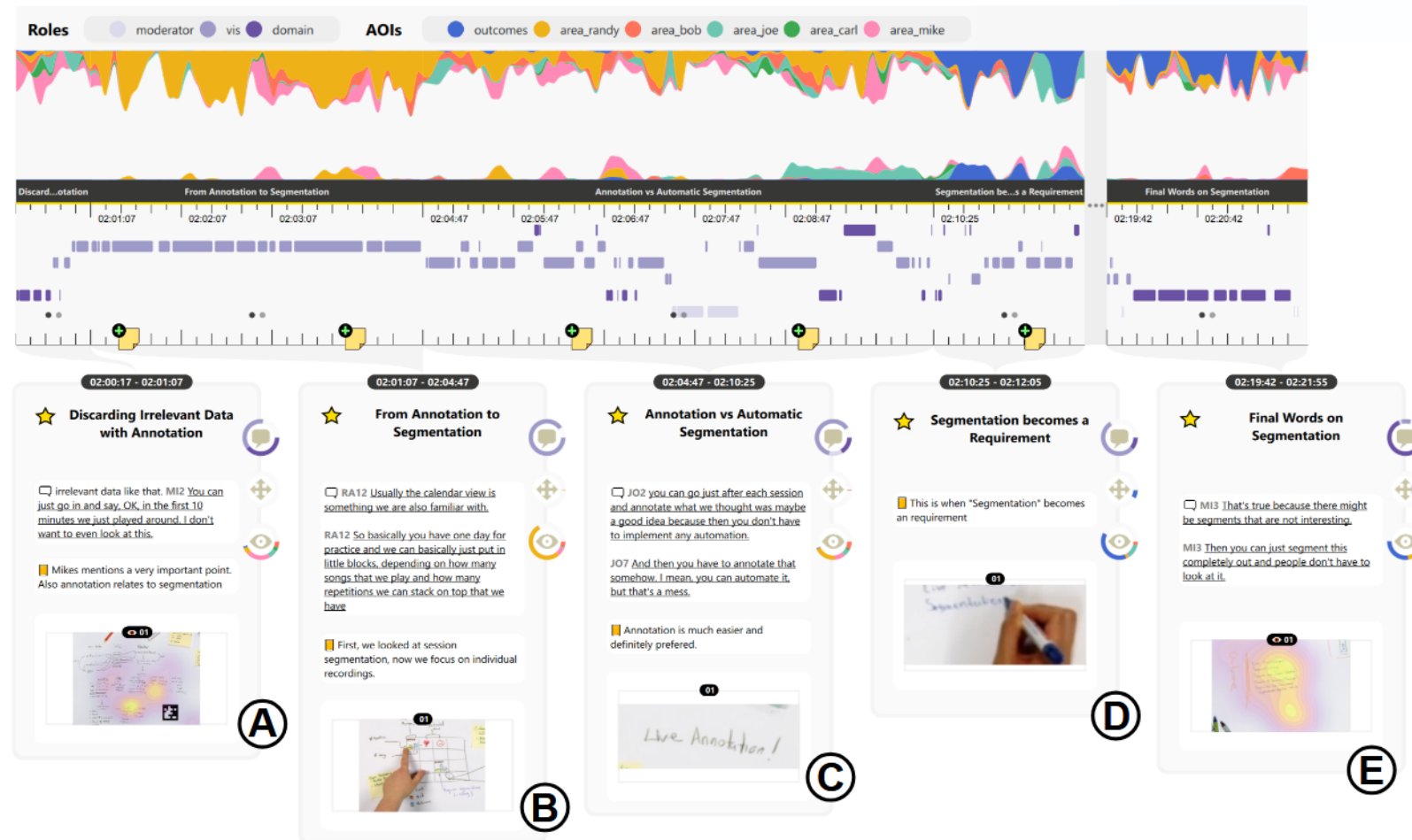


Fig. 9: Final analysis state related to the requirement *Segmentation* comprising the five topic segments (A)–(E). The streamgraphs show activity in Randy's area as he explains his visualization and adds the phrase "Live Annotation!" to his sketches.

Understand: Problem → Tasks → Requirements

- **Task elicitation:** What operations performed by the user must be supported?
- **Requirement gathering:** What are the necessary functionalities, capabilities, and characteristics of a system?

Tasks:

Lower level and more focused
on specific actions that users
need

Requirements:

Oriented towards reflecting
user intentions in terms of
system characteristics.



Understand: Problem → Tasks → Requirements

- **Task elicitation:** What operations performed by the user must be supported?
- **Requirement gathering:** What are the necessary functionalities, capabilities, and characteristics of a system?

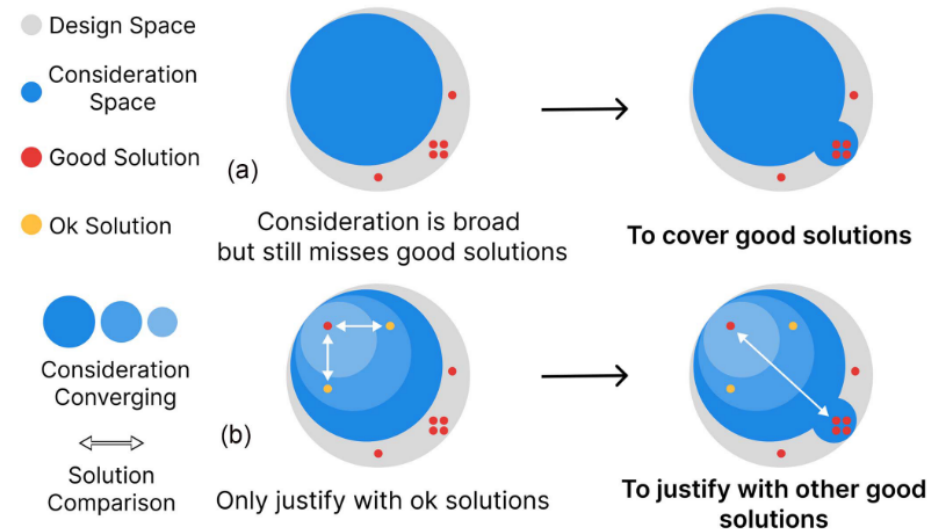


Fig. 1. An ill-positioned consideration space (e.g., with only *bad* and *ok* solutions) results in (a) the selection of *ok* solutions rather than *good* ones, and (b) the justification of current solution with *ok* solutions rather than *good* ones.

Wu et al., 2026

Understand: Problem → Tasks → Requirements

- **Task elicitation:** What operations performed by the user must be supported?
- **Requirement gathering:** What are the necessary functionalities, capabilities, and characteristics of a system?

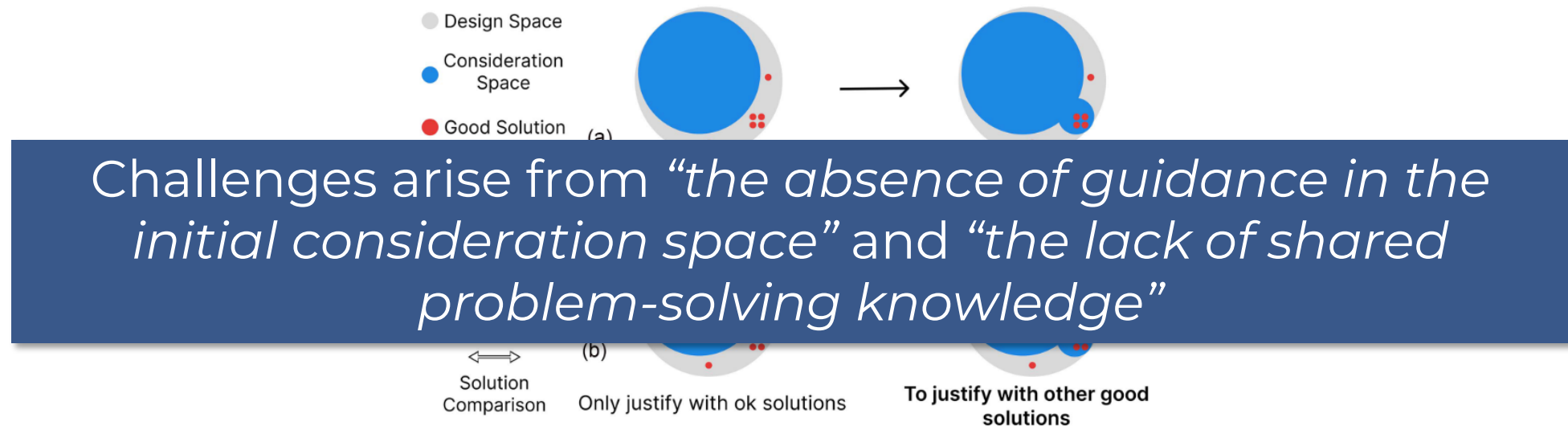


Fig. 1. An ill-positioned consideration space (e.g., with only *bad* and *ok* solutions) results in (a) the selection of *ok* solutions rather than *good* ones, and (b) the justification of current solution with *ok* solutions rather than *good* ones.

Wu et al., 2026

Understand: Problem → Tasks → Requirements

- Extract “material units” from each VA paper:
 - Segment each paper into discrete units describing requirements, data, and solutions.
- Code each unit using shared typologies.
 - Requirements
 - Data
 - Solutions
- Represent each paper as a knowledge graph and derive “solution paths”
 - **Requirement → Data** edges (“is imposed on”)
 - **Data → Solution** edges (“is utilized by”)
 - **Solution → Solution** edges (“as basis of”)

Understand: Problem → Tasks → Requirements

TABLE III
THE SOLUTION TYPOLOGY DERIVED FROM CODING SOLUTIONS

A) Data Manipulation				
<i>Eff\Tar</i>	<i>Augment</i>	<i>Reduce</i>	<i>Search</i>	<i>Modify</i>
	Real-Time Input	Excluding	Retrieval	Rectification
<i>Item</i>	User Input	Sampling	Similarity Calculation	Parameter Tuning
	Algorithmic Calculation	Dimensionality Reduction		
<i>Dimension</i>	Modeling	Feature Selection	Explainability	Wrangling
	Clustering & Grouping			

B) Visualization							
Composite							
Juxtaposition			Overlay				Nesting
Similar		Unsimilar	Coordinate system-related		Coordinate system-unrelated		Nesting
<i>Nonsymmetrical</i>	<i>Symmetrical</i>	<i>Continuous</i>	<i>Sharing</i>	<i>Providing</i>	<i>With connections</i>	<i>W/O connections</i>	
Repetition	Mirror	Stack	Co-axis	Coordinate	Annotation	Large panel	
Non-composite							
With layout			Without Layout				
Aligned		Flexible			Basics		

C) Interaction									
Filtering	Selecting	Abstract/Elaborate	Overview and Explore	Connect/Relate	Reconfigure	Encode	History	Extraction of Features	Participation/Collaboration
									Gamification

The typology combines previous typologies on visualization [6] and interaction [7], while introducing a new solution type of a) data manipulation. The data manipulations are organized by two axes: *effect* (horizontal) ranges from augment to modify and denotes the expected outcomes when applying the manipulation; *target* (Vertical) denotes the entities on which the manipulation operates.

Wu et al., 2026

Understand: Problem → Tasks → Requirements

A **solution path** is then a path from a requirement node to a “leaf” solution node (zero outdegree), meant to capture one atomic, coherent way that paper addresses the requirement.

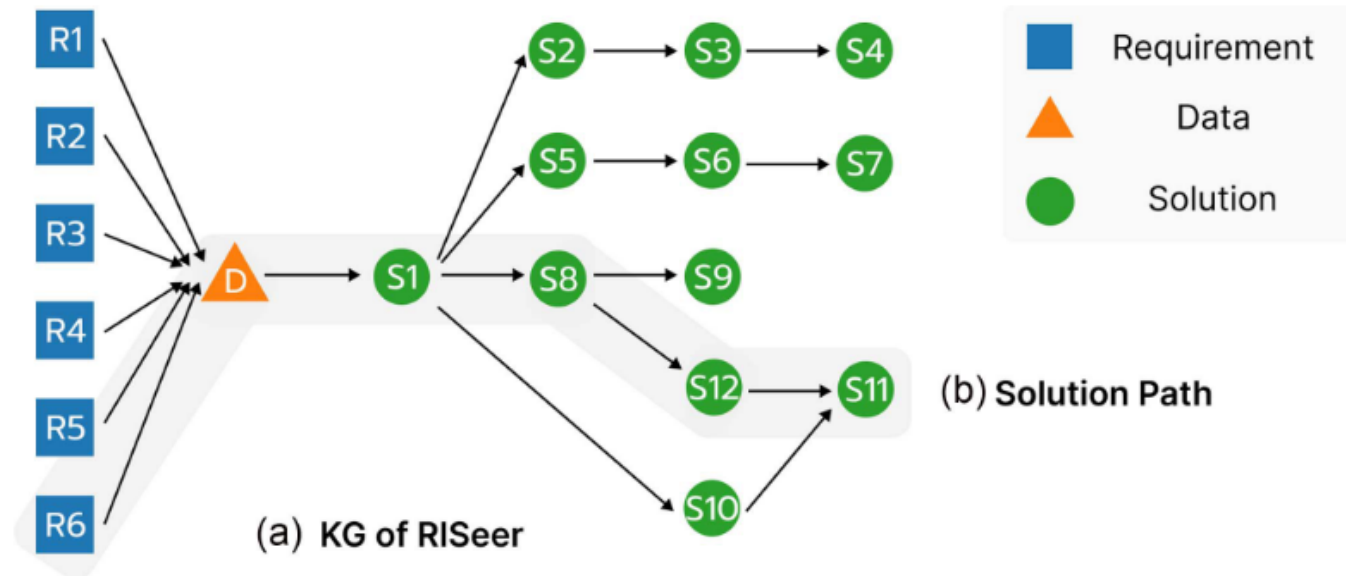


Fig. 6. We represent the problem-solving knowledge in VA research as a knowledge graph of requirements, data, and solutions. (a) An example of KG of RISeer; (b) An instance of solution path.

Understand: Problem → Tasks → Requirements

How it can be used in practice?

1. “Design search” for requirement → candidate solution strategies
 - Given a requirement (and data), we can traverse the base to see what solution components and sequences have been used in prior VA work.
 - Wu et al. argue that rather than a single recommendation, their work can support a **panoramic view** of the landscape of VA solutions.
2. Evidence for design justification
 - Used as evidence, showing how requirements have been addressed through different components.
3. Deriving reusable “problem-driven design patterns”
 - Frequently occurring subgraphs / paths are treated **as design patterns**. A pattern library where patterns are anchored in many instances from literature.
4. Tooling: a knowledge base for teams
 - A graph database that supports queries such as “What interaction types frequently appear downstream of this visualization?” or “What are the sparse areas (few paths) suggesting research gaps?”



Ideate: Data exploration → Sketching → Prototyping

- Goal: Produce multiple plausible solution directions.
- Pain points:
 - The space of possible solutions is huge.
 - Early design gets anchored to whatever you explore first (local optimum).
 - Iteration trash: ideas bounce around without tracking what changed and why; teams repeat mistakes.
- **AI opportunities:**
 - **Idea generation:**
Propose alternative visualizations.
 - **Rapid data profiling:**
Auto-summarize dataset types, quality issues, etc.
 - **Visualization iteration:**
Provide feedback for visualizations:
 - **Summarization:**
Summarize notes and transcripts into records of options and decisions made.

Ideate: Data exploration → Sketching → Prototyping

- Goal: Produce multiple plausible solution directions.
- Pain points:
 - The space of possible solutions is huge.
 - Early design gets anchored to whatever you explore first (local optimum).
 - Iteration trash: ideas bounce around without tracking what changed and why; teams repeat mistakes.



Ideate: Data exploration → Sketching → Prototyping

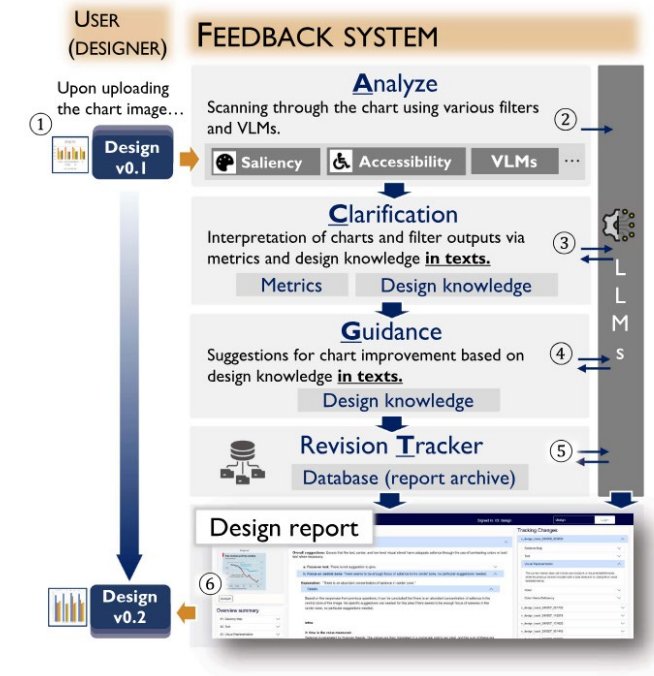
- Shin et al. propose Visualizationary, an LLM- and VLM-based tool to support visualization iteration.
- Design knowledge is incorporated as preambles in the prompt.

$$T_{acg} = [Q] +$$

“Solve the problem based on this guideline:” +
[cond] + [filter-suggestions].

$$T_t = [Q] +$$

“Here are details about the current version:” +
[curr-output] + [curr-interpretations]
+ “Here are details about the previous version:” +
[prev-output] + [prev-interpretations].



Shin et al., 2026

Ideate: Data exploration → Sketching → Prototyping

- **Q:** Analyze the visual salience on text. Provide interpretations in 2 sentences.
- **CONDITIONS:** If no visual salience is detected, say: "No salience is detected in the chart image."
- **FILTER-SUGGESTIONS** (design knowledge, optional): If textual elements lack salience, then increase contrast, increase font weight/size, or reduce competing salience from decorative elements.



Ideate: Data exploration → Sketching → Prototyping

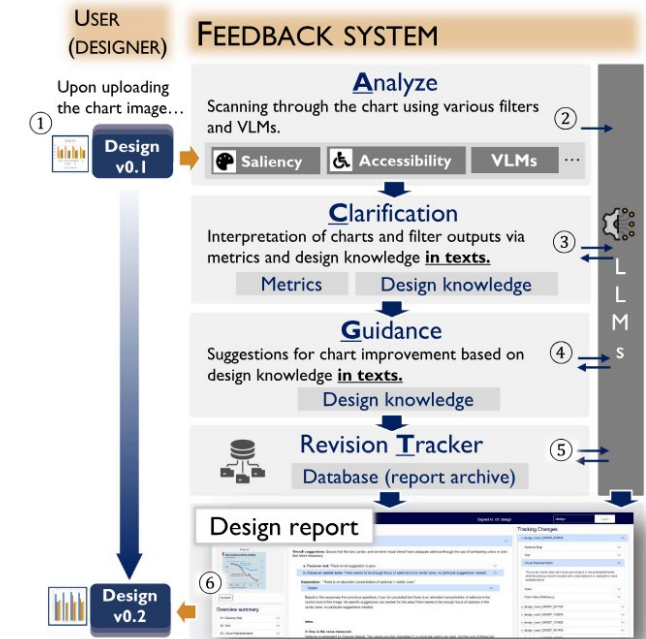
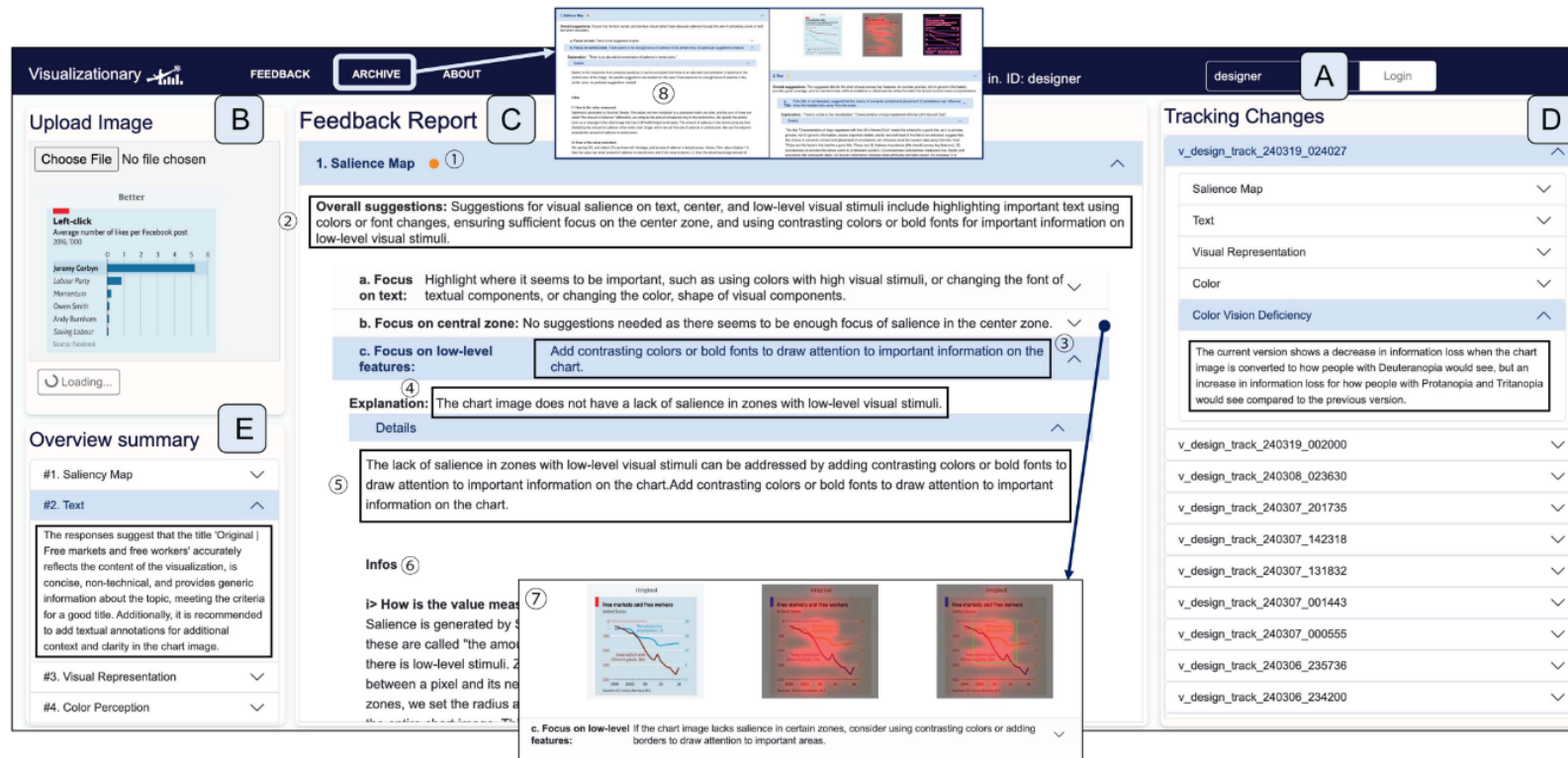


Fig. 2. Visualizationary interface. Example of an interactive report on a visualization artifact generated using our novel analyze-clarify-guide-track (ACGT) framework. The image is a depiction of our system, Visualizationary where a new chart image (Fig. 1(B)) is being updated while the current report is about the past visualization image. The report is generated by automatically analyzing (“analyze”) the artifact using an expandable collection of image filters grouped into categories: saliency, text, visual representation, color, and accessibility. After reporting results from each image filter (center column), the report then interprets (“clarify”) the findings using natural language understandable to a novice visualization designer. This is followed by natural language suggestions (“guide”) on how to address weaknesses in the design. Finally, the right side of the report shows changes over time (“track”) as the designer iteratively refines the artifact.

Shin et al., 2026

Make: Implementation → Integration → Iteration

- Goal: Turn sketches and prototypes into a working VA system.
- Pain points:
 - Architecture drift: implemented system diverges from the intended design.
 - Integration complexity: wiring components is complex.
 - Regression risk: every change breaks something else.
- **AI opportunities:**
 - **Code scaffolding:**
Generate and enforce consistent patterns.
 - **Data pipeline assistance:**
Suggest transformations, schemas.
 - **Documentation as-you-build:**
Auto-generate API docs, onboarding / tutorial content.

Make: Implementation → Integration → Iteration

- Goal: Turn sketches and prototypes into a working VA system.
- Pain points:
 - Architecture drift: implemented system diverges from the intended design.
 - Integration complexity: wiring components is complex.
 - Regression risk: every change breaks something else.



Make: Implementation → Integration → Iteration

6162

IEEE TRANSACTIONS ON VISUALIZATION AND COMPUTER GRAPHICS, VOL. 31, NO. 9, SEPTEMBER 2025

LightVA: Lightweight Visual Analytics With LLM Agent-Based Task Planning and Execution

Yuheng Zhao[✉], Junjie Wang, Linbing Xiang[✉], Xiaowen Zhang, Zifei Guo[✉], Cagatay Turkey[✉], Yu Zhang[✉], and Siming Chen[✉]

Abstract—Visual analytics (VA) requires analysts to iteratively propose analysis tasks based on observations and execute tasks by creating visualizations and interactive exploration to gain insights. This process demands skills in programming, data processing, and visualization tools, highlighting the need for a more intelligent, streamlined VA approach. Large language models (LLMs) have recently been developed as agents to handle various tasks with dynamic planning and tool-using capabilities, offering the potential to enhance the efficiency and versatility of VA. We propose LightVA, a lightweight VA framework that supports task decomposition, data analysis, and interactive exploration through human-agent collaboration. Our method is designed to help users progressively translate high-level analytical goals into low-level tasks, producing visualizations and deriving insights. Specifically, we introduce an LLM agent-based task planning and execution strategy, employing a recursive process involving a planner, executor, and controller. The planner is responsible for recommending and decomposing tasks, the executor handles task execution, including data analysis, visualization generation and multi-view composition, and the controller coordinates the interaction between the planner and executor. Building on the framework, we develop a system with a hybrid user interface that includes a task flow diagram for monitoring and managing the task planning process, a visualization panel for interactive data exploration, and a chat view for guiding the model through natural language instructions. We examine the effectiveness of our method through a usage scenario and an expert study.

Index Terms—Visual analytics, task planning, large language model agent, mixed-initiative interaction.

I. INTRODUCTION

VISUAL analytics (VA) deciphers complex datasets with data mining and interactive visualizations [1], [2]. However, building and using a VA system can be a costly endeavor that encompasses several main stages: goal understanding, task decomposition, data modeling, and visualization creation to

discover insights. A key challenge is that this process is iterative, requiring continual refinement based on evolving needs [3]. Different tasks necessitate various data analysis and visualization methods to form a VA system. While using the system, tasks may evolve based on the insights gained, necessitating ongoing iterations until the analytical goals are achieved [4]. Consider a scenario where the goal is to identify high-risk events from social media data. Users may first need an overview from different perspectives, such as the distribution of keywords over time or changes in sentiment for risk analysis. If outliers are identified, users may need further details, such as spatial distribution or entity relationships. In this process, the task space is broad and fluid, requiring efficient task planning and method implementation.

Recent research focuses on data-driven or natural language-based visual data exploration, with an emphasis on automatic visualization generation [5], [6] or insight mining [7], [8]. Large Language Models (LLMs) present a potential for data analysis, supporting dynamic task planning and lower development costs across various scenarios. The reasoning abilities that enable autonomous planning and execution of analytical tasks [9], [10], [11], while code generation capability support creating insightful visualizations efficiently [12], [13], [14]. Additionally, their broad knowledge base makes LLMs versatile tools capable of adapting to diverse data analysis contexts [15], [16]. LEVA [17] integrates LLMs in VA systems to recommend insights for a given task but still cannot generate visualization and data modeling methods adapted to tasks. There is a lack of approaches supporting task planning, VA methods implementation, and interactive analysis with human-agent collaboration.

This paper introduces *LightVA*, a lightweight VA framework with agent-based task planning. The term “lightweight” refers to the framework’s focus on reducing the cost of development and using VA systems. Using LLM agents to aid the task planning and execution process. The framework builds upon multi-level relationships, translating high-level goals to low-level tasks and deriving insights through data mining and interactive visualizations. Specifically, the framework employs a recursive process that includes a planner, executor, and controller, which dynamically accommodates task complexity. The planner is responsible for task decomposition, the executor handles task execution, including visualization generation and data analysis, and the controller orchestrates the executor and planner and manages whether tasks continue to be decomposed. We develop a system based on the framework that provides a chat view to support

1077-2626 © 2024 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on February 23, 2026 at 20:16:44 UTC from IEEE Xplore. Restrictions apply.

IEEE TRANSACTIONS ON VISUALIZATION AND COMPUTER GRAPHICS, VOL. 32, NO. 1, JANUARY 2026

1197

VA-Blueprint: Uncovering Building Blocks for Visual Analytics System Design

Leonardo Ferreira, Gustavo Moreira, and Fabio Miranda

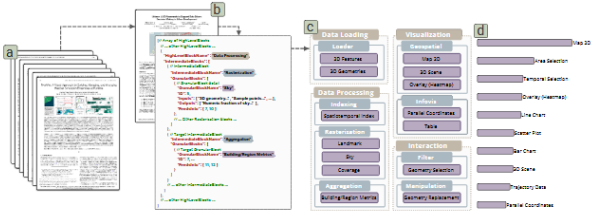


Fig. 1: VA-Blueprint provides a structured way to uncover and organize the fundamental building blocks used in visual analytics (VA) systems. (a) The process begins with the curation of a corpus of VA system research papers. (b) From each paper, core components and their relationships are systematically extracted and structured into a formal, hierarchical blueprint (represented as JSON), detailing the system’s components and their dependencies. (c) This blueprint categorizes components across multiple levels of abstraction: high-level stages (e.g., Data Processing), intermediate groups (e.g., Visualization), and specific granular blocks (e.g., Sky Computation). (d) The final knowledge base comprises 101 machine-readable blueprints encoding component hierarchies, and data and interaction dependencies.

Abstract—Designing and building visual analytics (VA) systems is a complex, iterative process that requires the seamless integration of data processing, analytics capabilities, and visualization techniques. While prior research has extensively examined the social and collaborative aspects of VA system authoring, the practical challenges of developing these systems remain underexplored. As a result, despite the growing number of VA systems, there are only a few structured knowledge bases to guide their design and development. To tackle this gap, we propose VA-Blueprint, a methodology and knowledge base that systematically reviews and categorizes the fundamental building blocks of urban VA systems, a domain particularly rich and representative due to its intricate data and unique problem sets. Applying this methodology to an initial set of 20 systems, we identify and organize their core components into a multi-level structure, forming an initial knowledge base with a structured blueprint for VA system development. To scale this effort, we leverage a large language model to automate the extraction of these components for other 81 papers (completing a corpus of 101 papers), assessing its effectiveness in scaling knowledge base construction. We evaluate our method through interviews with experts and a quantitative analysis of annotation metrics. Our contributions provide a deeper understanding of VA systems’ composition and establish a practical foundation to support more structured, reproducible, and efficient system development. VA-Blueprint is available at urbantk.org/va-blueprint.

Index Terms—Visual analytics, large language models, knowledge base, system development, urban visual analytics

1 INTRODUCTION

Visual analytics (VA) systems help users make sense of complex data by combining analytics with interactive visualizations. Their usefulness has been demonstrated across diverse domains, including biology [27], healthcare [28], urban planning [17], and climate science [20]. However, building VA systems remains a highly complex and iterative process. System builders must integrate data processing techniques, analytics capabilities, and visualization strategies while ensuring interpretability for domain experts and maintaining efficiency to support interactivity. Despite their significance and inherent challenges, VA

system development is often approached in an ad-hoc manner, with minimal reuse of existing components. Given the necessity to address the unique analytical and visualization needs of specific domains, these systems are usually bespoke solutions, leading to fragmented development efforts, making them difficult to extend, adapt, or reuse across projects. This tension, between the need for tailored solutions and the demand for scalable, extensible systems, presents a fundamental challenge in VA system development [12, 58]. On one hand, VA systems must be grounded by domain-specific data, tasks, and analytical workflows: a visualization technique that works well for transportation may be inadequate for urban planning, just as an analytical model for climate science may not be applicable to public health. On the other hand, as VA systems become increasingly complex, the lack of systematic and reusable components hampers the design process and increases the effort needed for development.

Currently, there is a lack of practical knowledge bases that document the building blocks of VA systems. Initial steps have produced valuable taxonomies for visualization tasks and techniques [5]. More recently, new initiatives have emerged to catalog and structure design elements

Received 31 March 2025; revised 1 July 2025; accepted 15 July 2025.
Date of publication 4 December 2025; date of current version 3 February 2026.
Digital Object Identifier 10.1109/TVCG.2025.3634809

• Leonardo Ferreira, Gustavo Moreira, and Fabio Miranda are with the University of Illinois Chicago. E-mail: {lferrei1, gmorei1, fabiomir}@uic.edu.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on February 16, 2026 at 22:46:13 UTC from IEEE Xplore. Restrictions apply. Personal use is permitted, but republication/redistribution requires IEEE permission. See <https://www.ieee.org/publications/rights/index.html> for more information.



urbantk.org



electronic
visualization
laboratory



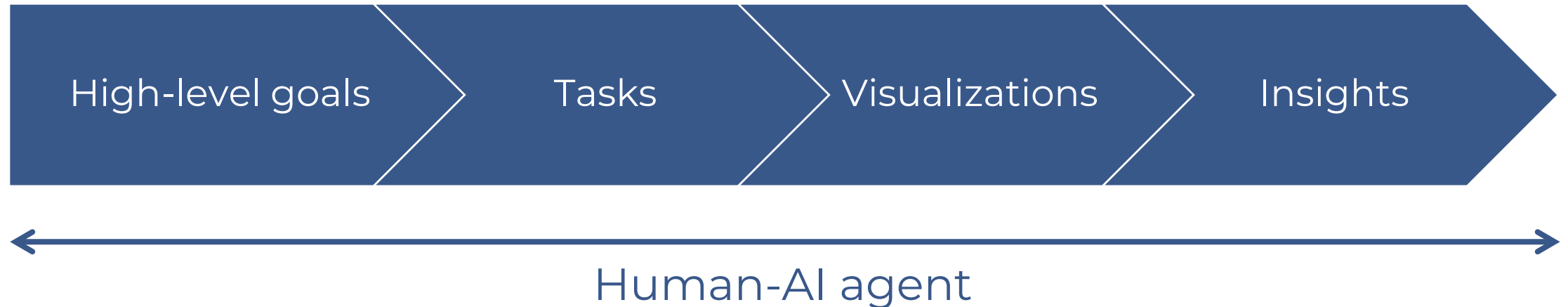
UNIVERSITY OF
ILLINOIS CHICAGO

LightVA

- Building and using a VA system can be a costly endeavor.
 - Real barrier: requires programming, data processing, and vis expertise.
- Encompasses several stages:
 1. goal understanding
 2. task decomposition
 3. data modeling
 4. visualization creation
- This process is iterative, requiring continual refinement based on evolving needs.

LightVA

- AI agents suggest a new path forward: **planning + tool use to streamline VA**
- LightVA: a lightweight VA that unifies:
system construction + system use



LightVA: Challenges

- **C1:** Accommodating diverse analysis requirements
 - Data analysts often face challenges of navigating a vast exploration space.
 - Forming hypotheses is complex.
 - They have to figure out the connections between tasks and insights in their mind and propose tasks in the next few steps until achieve the goal.
- **C2:** Developing visualization and data mining tools is time consuming
 - Analysts often lack proficiency in developing and managing VA systems.
 - When tasks involve multiple visualizations, these must be integrated into a linked view.
 - This process requires substantial knowledge of visualization principles and coding skills.

LightVA: Design requirements

- **R1:** Adaptive task planning: tasks should adapt to goals, data, and evolving results (C1)
- **R2:** Flexible visualization generation: data identification/transform + viz choice + interactive views (C2)
- **R3:** Automatic insight generation: complete code-based analyses + highlight key findings (C2)
- **R4:** Multiple view composition: keep prior results, allow merging views + reduce cognitive load (C2)
- **R5:** Intuitive analysis process: make task–viz–insight–progress relationships legible; support human–agent interaction (C1, C2)

LightVA: Conceptual framework

- “Lightweight”: **lower expectations** for both *building* VA systems and *using* them
- Key primitives:
 - Goal: high-level purpose statement
 - Task: executable aspect of goal, defined by attributes (type, variables, method, progress)
- **Agent:** Task planning and execution
- **Human:** Refinement

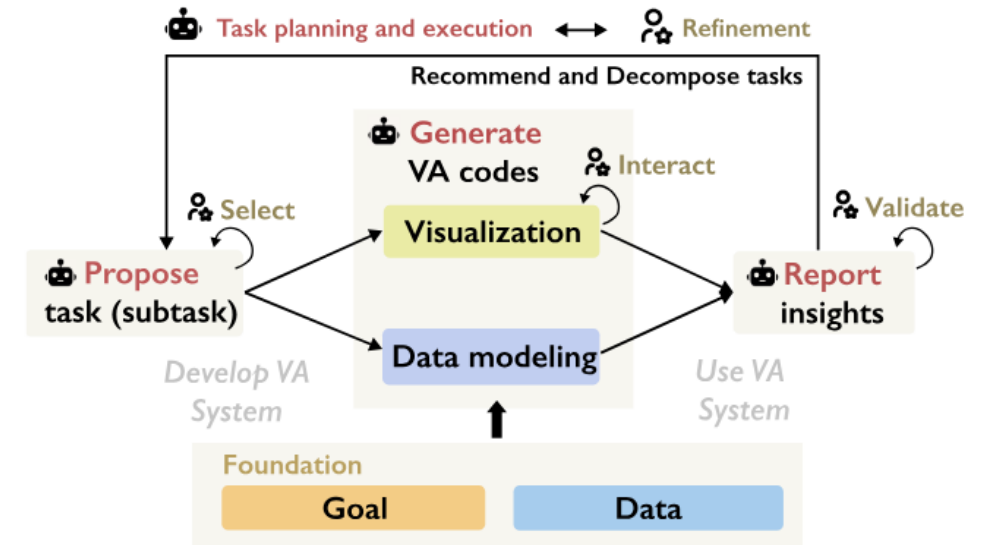
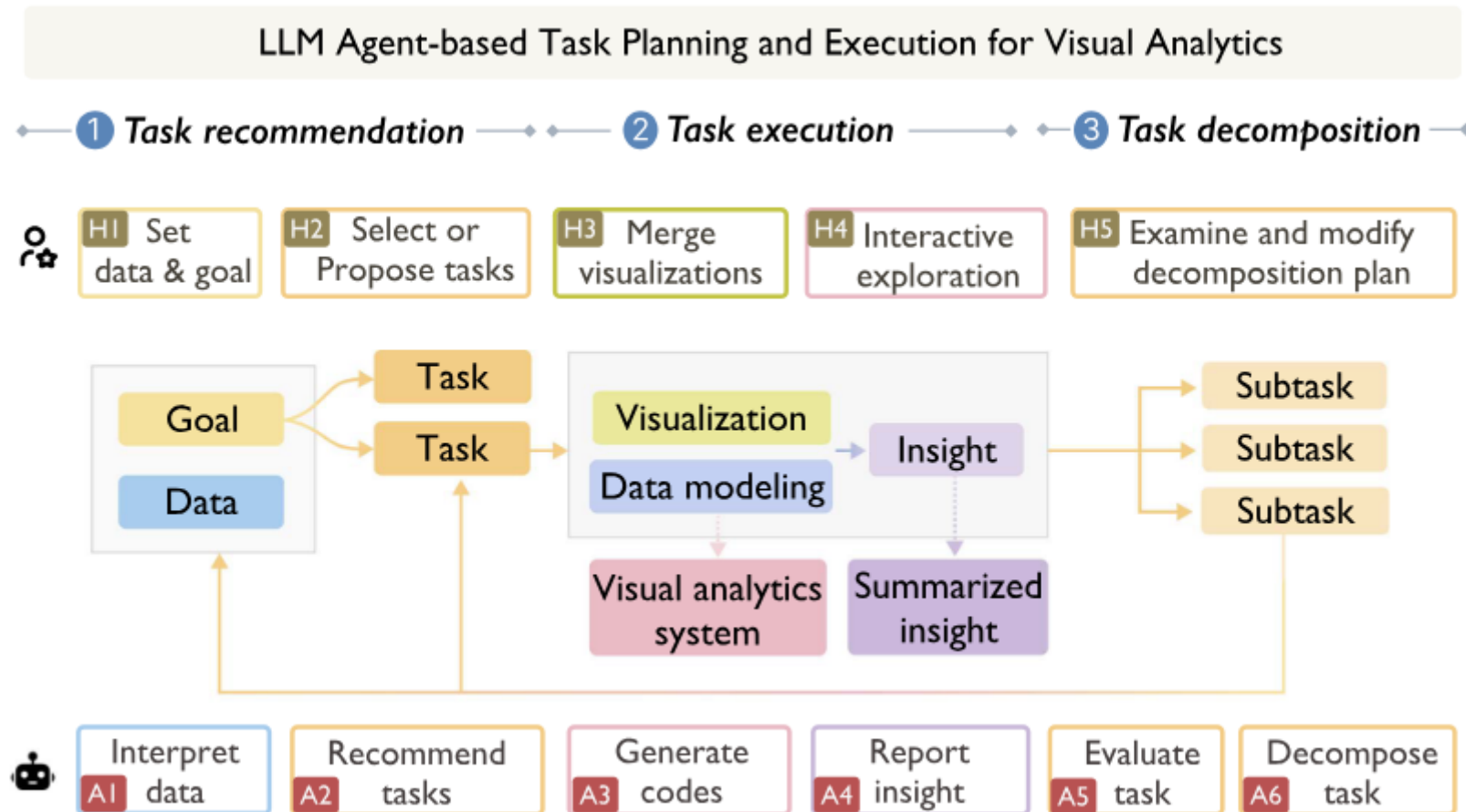
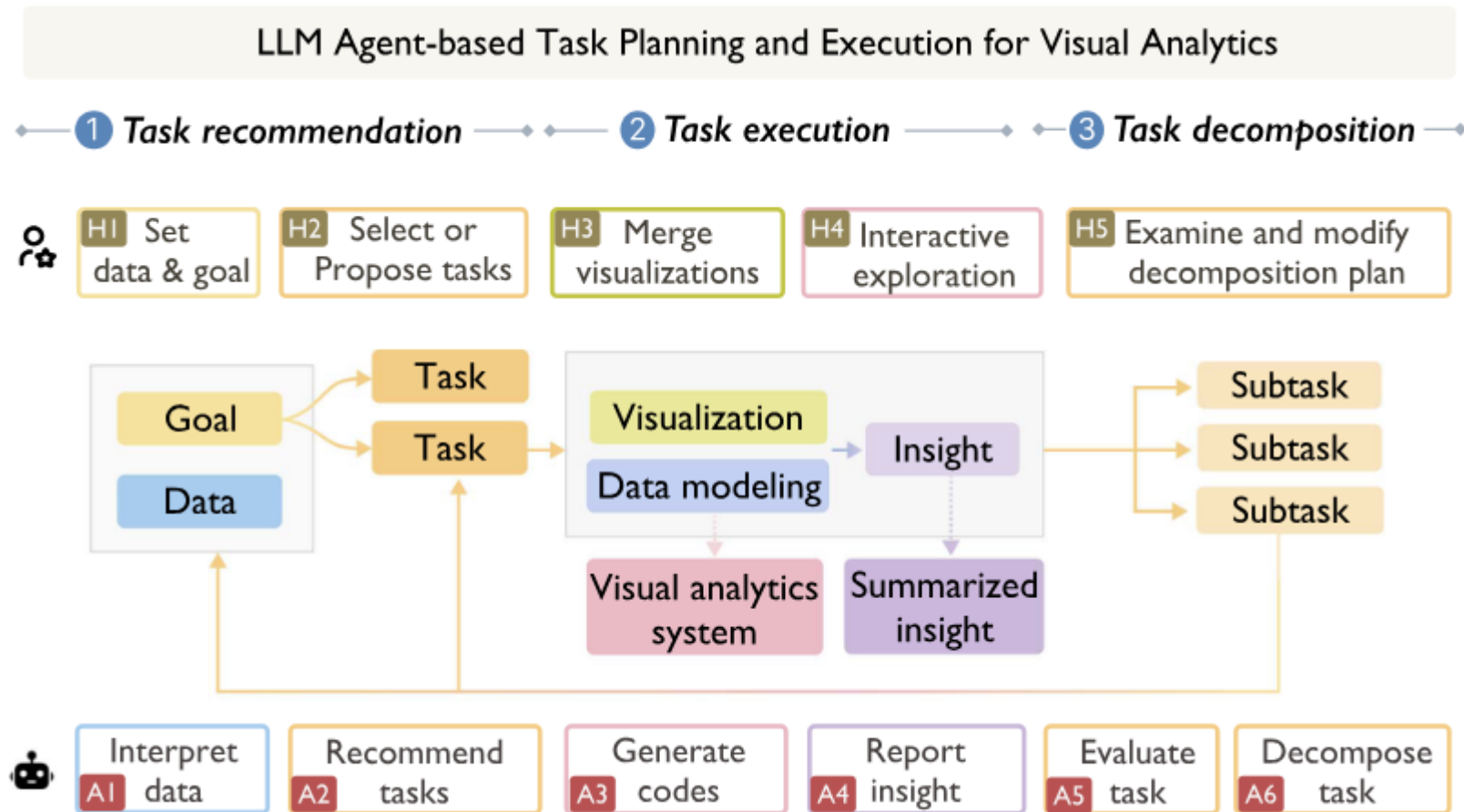


Fig. 1. The illustration of conceptual framework in LightVA. The agent creates the VA system for analysis by task planning and execution, while the user uses the created VA system and refines agent outputs.

LightVA: Collaboration workflow



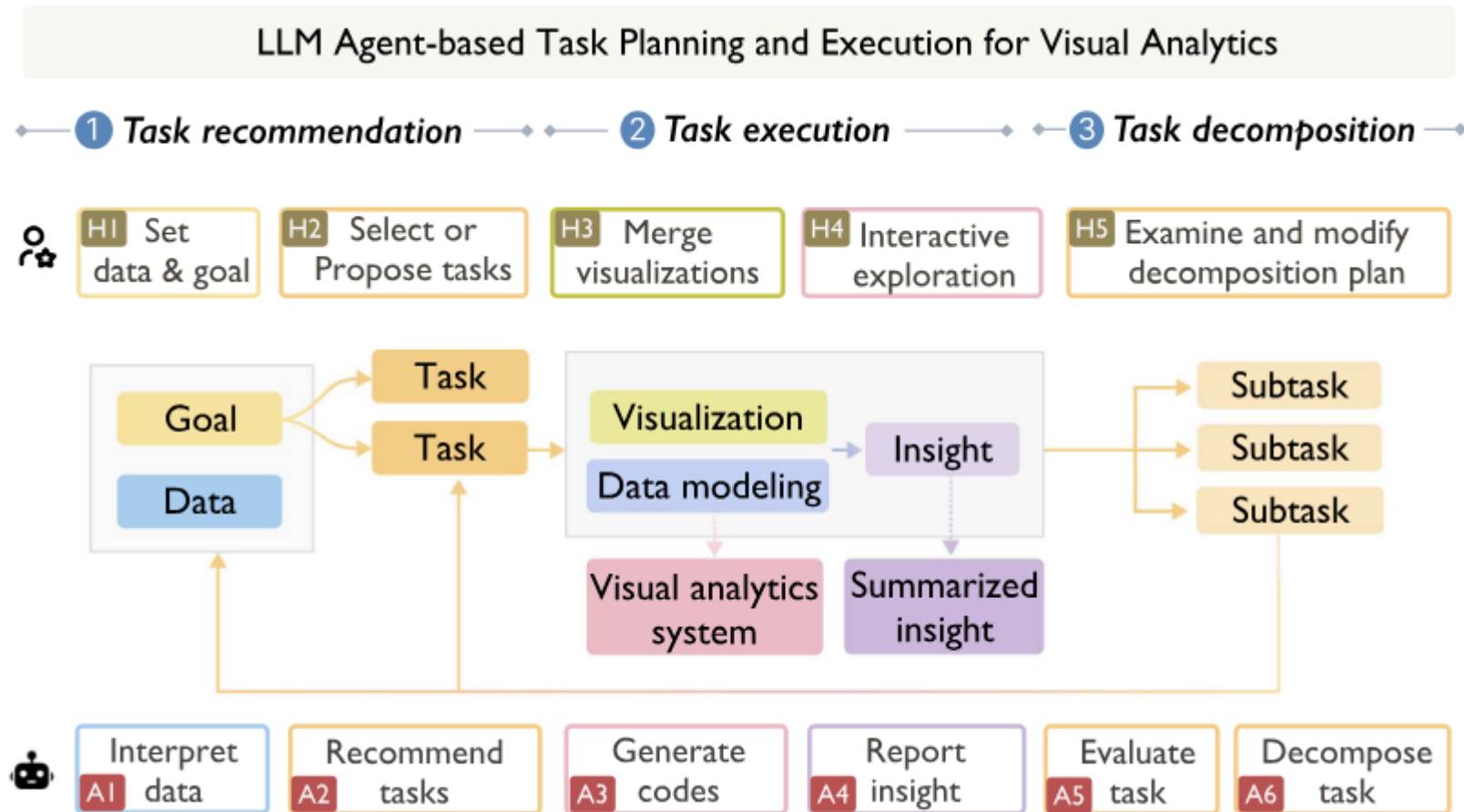
LightVA: Collaboration workflow



Stage 1: Task recommendation

- Human: upload data, state goal, accept / modify tasks
- Agent: interpret data, propose tasks

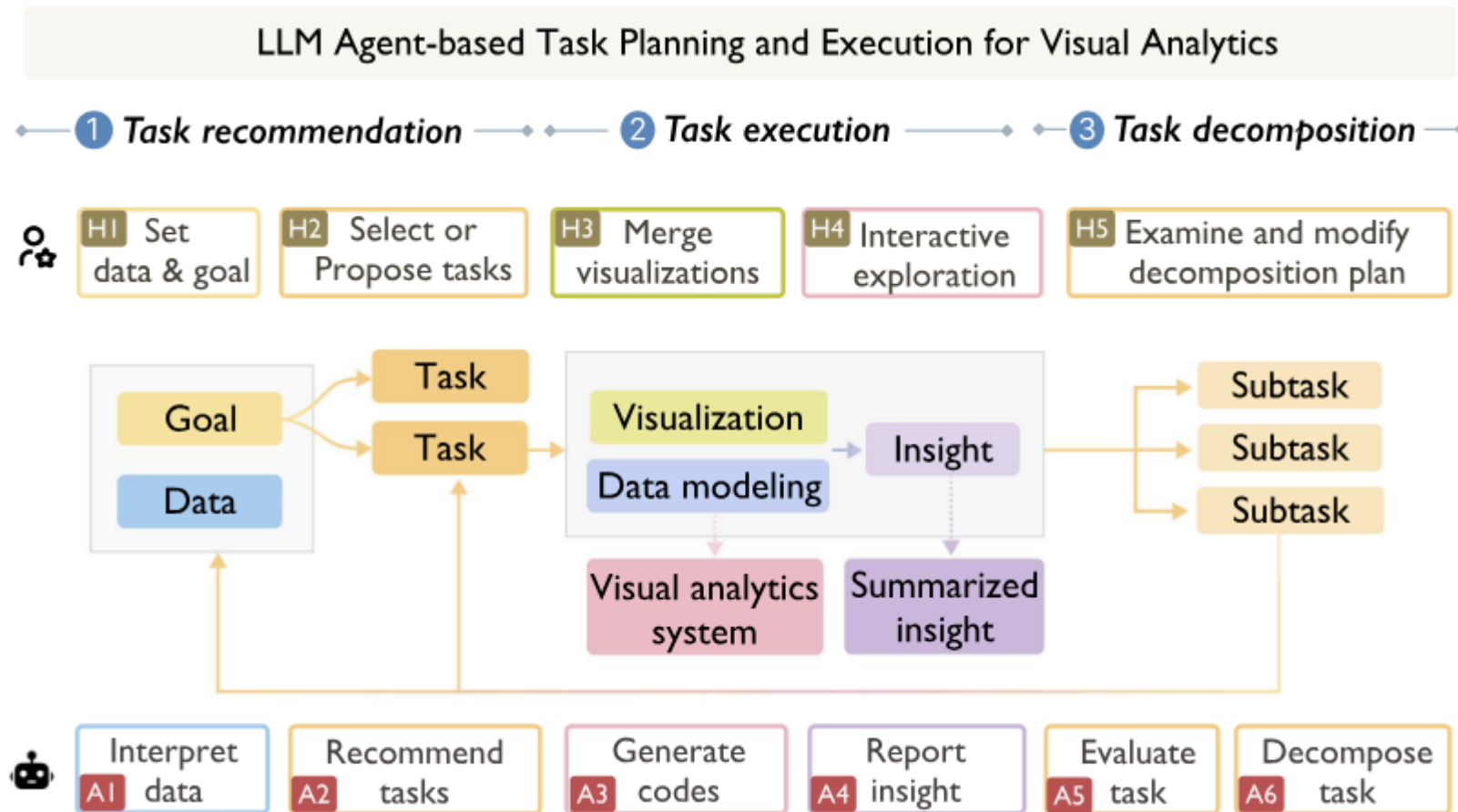
LightVA: Collaboration workflow



Stage 2: Task execution

- Human: merge views, explore linked view
- Agent: generate code for modeling and visualization, summarize insights

LightVA: Collaboration workflow



Stage 3: Task decomposition

- Human: review plan
- Agent: assess results, propose decomposition plan

LightVA: Collaboration workflow

LLM Agent-based Task Planning and Execution for Visual Analytics

1 Task recommendation → 2 Task execution → 3 Task decomposition →



H1 Set data & goal

H2 Select or Propose tasks

H3 Merge visualizations

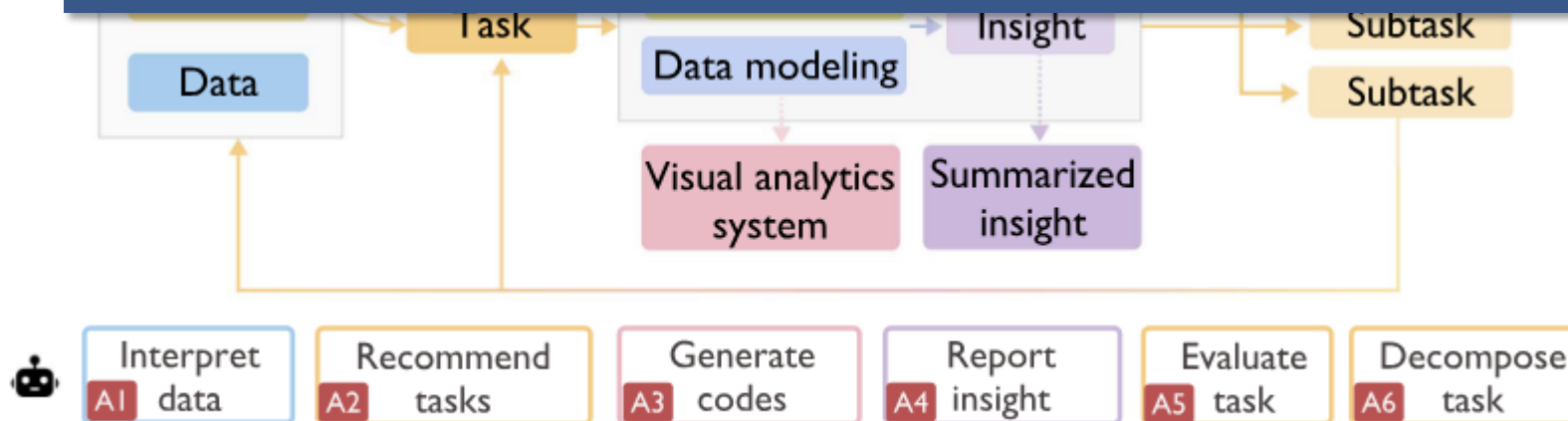
H4 Interactive exploration

H5 Examine and modify decomposition plan

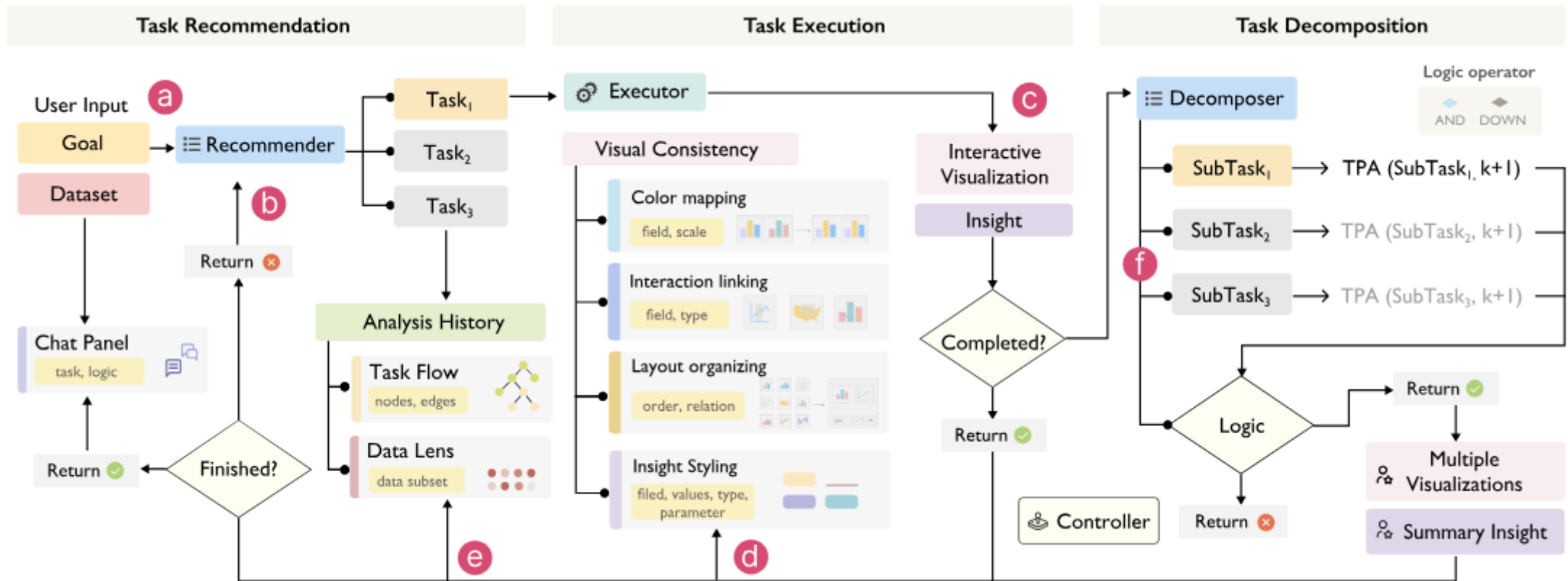
Stage 3: Task decomposition

- Human: review plan
- Agent: assess results,

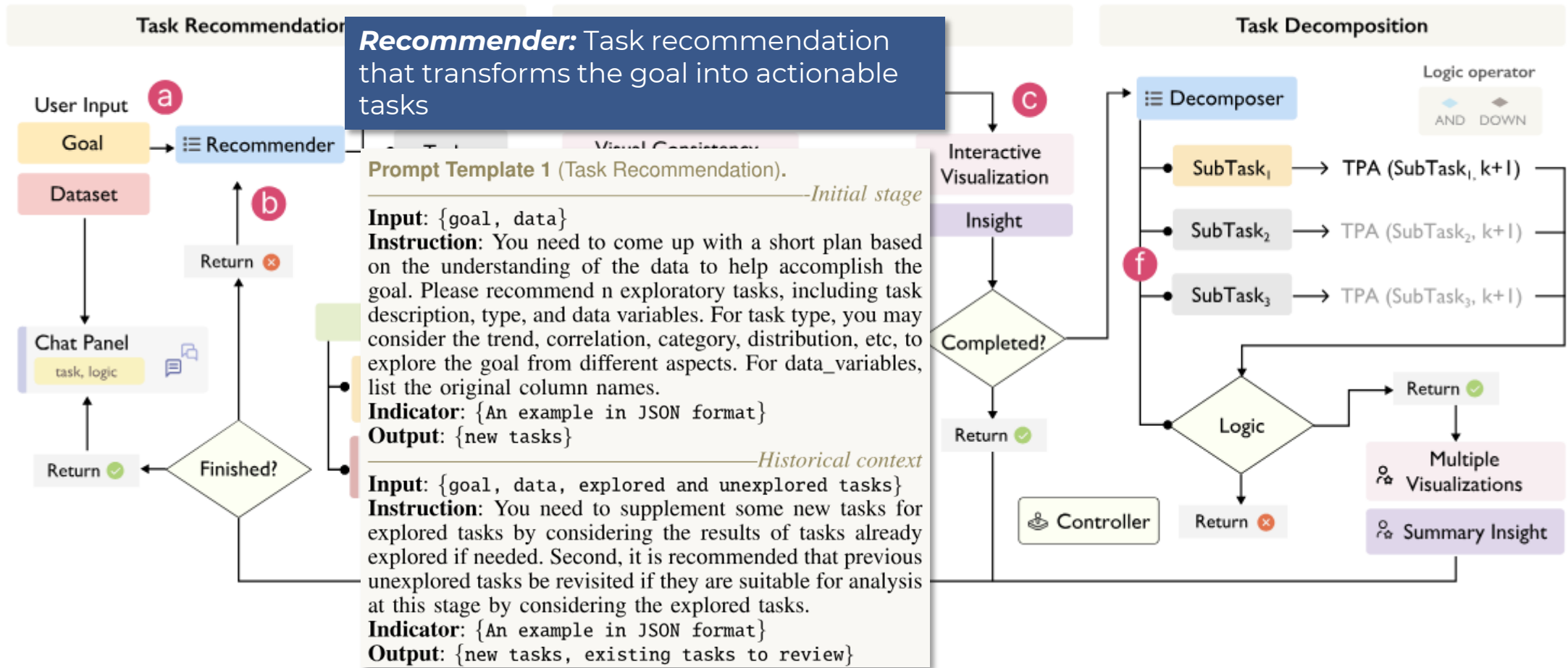
How is this operationalized into a system?



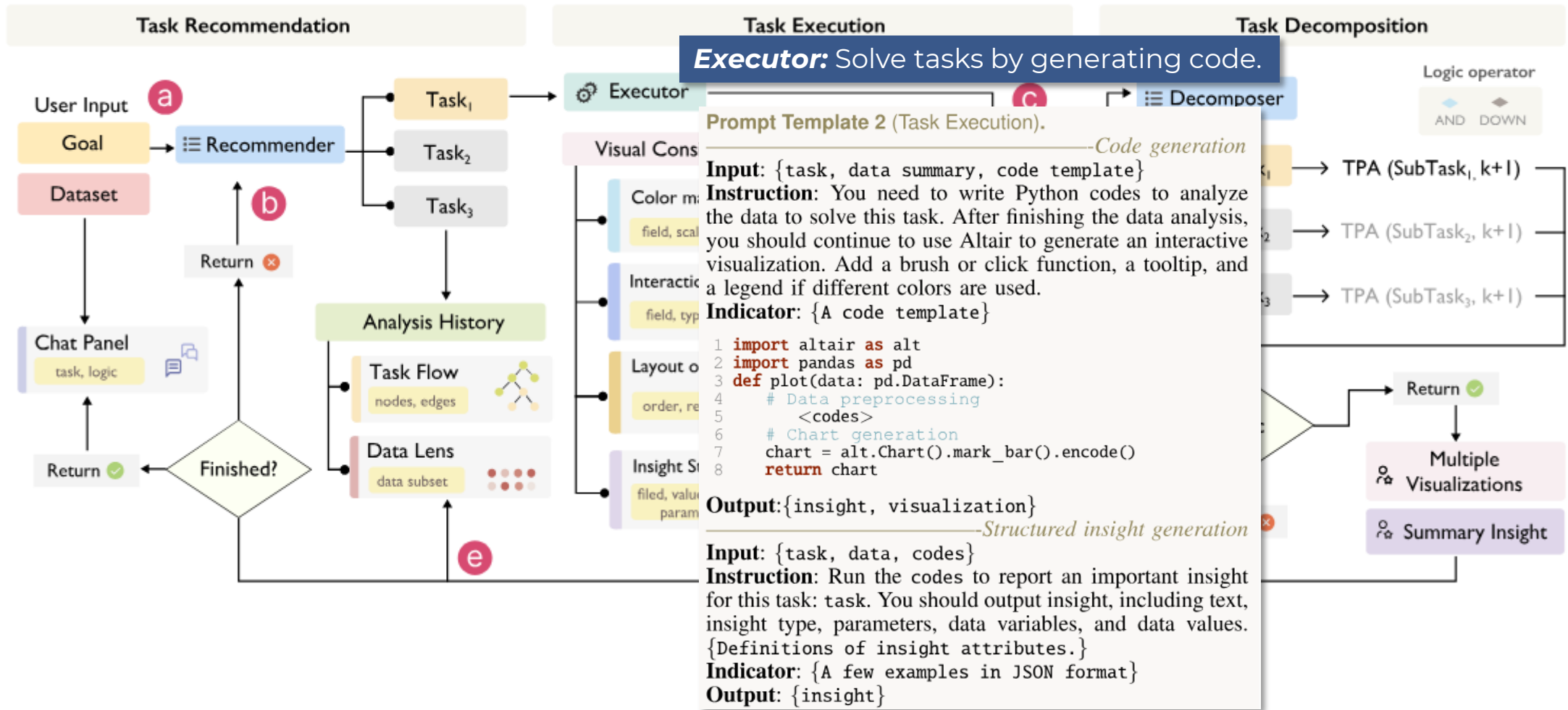
LightVA: System architecture



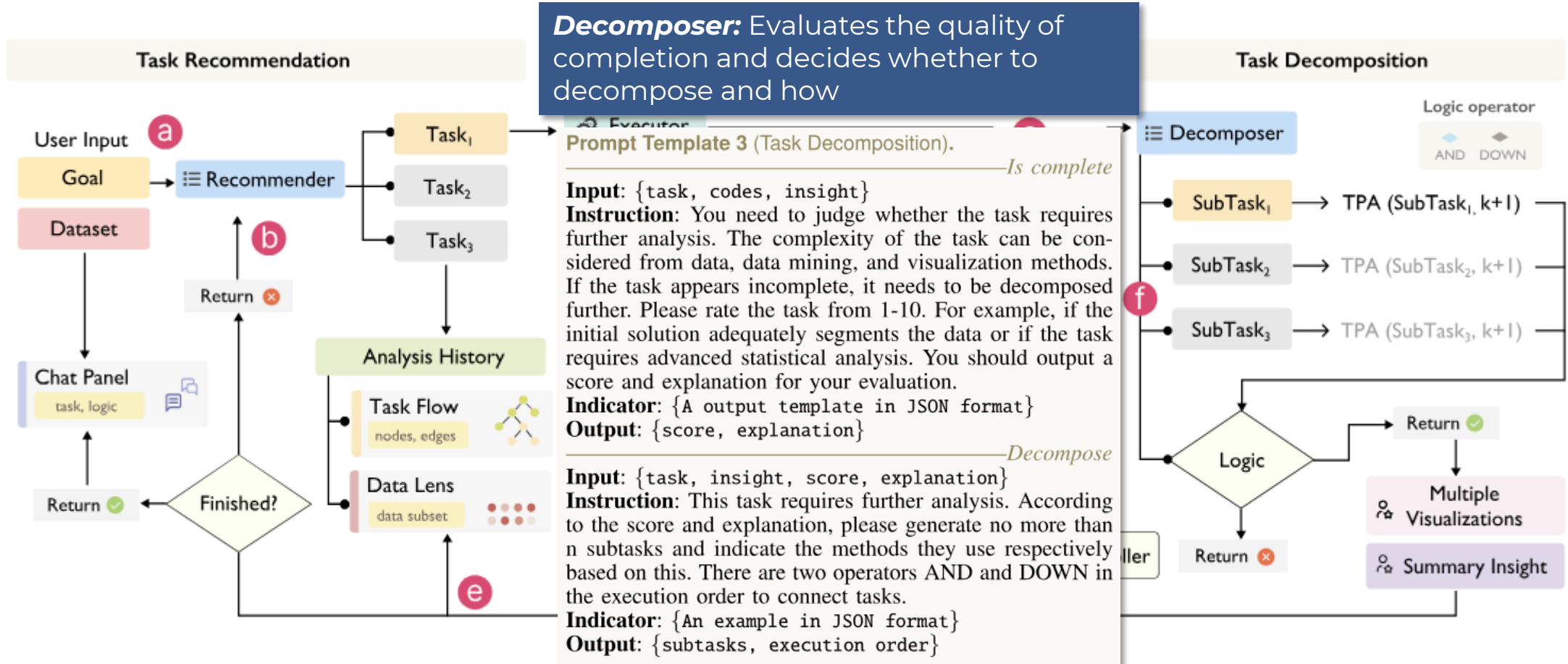
LightVA: System architecture



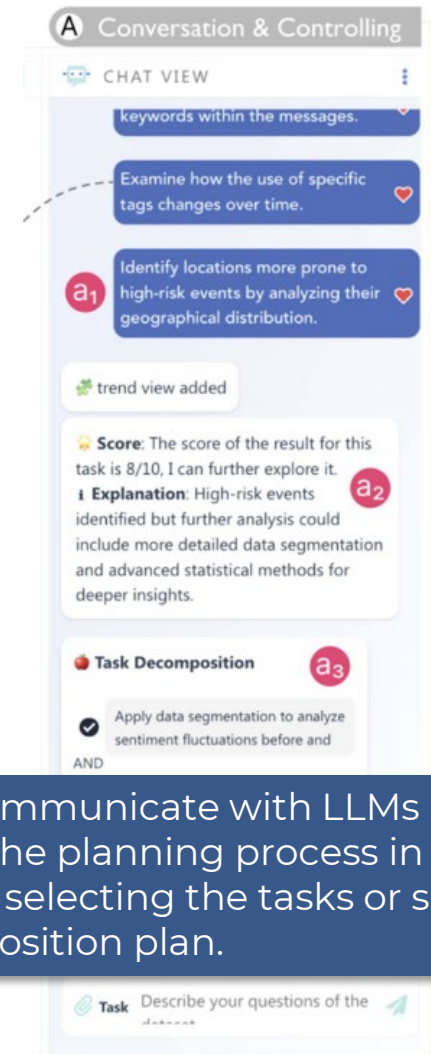
LightVA: System architecture



LightVA: System architecture

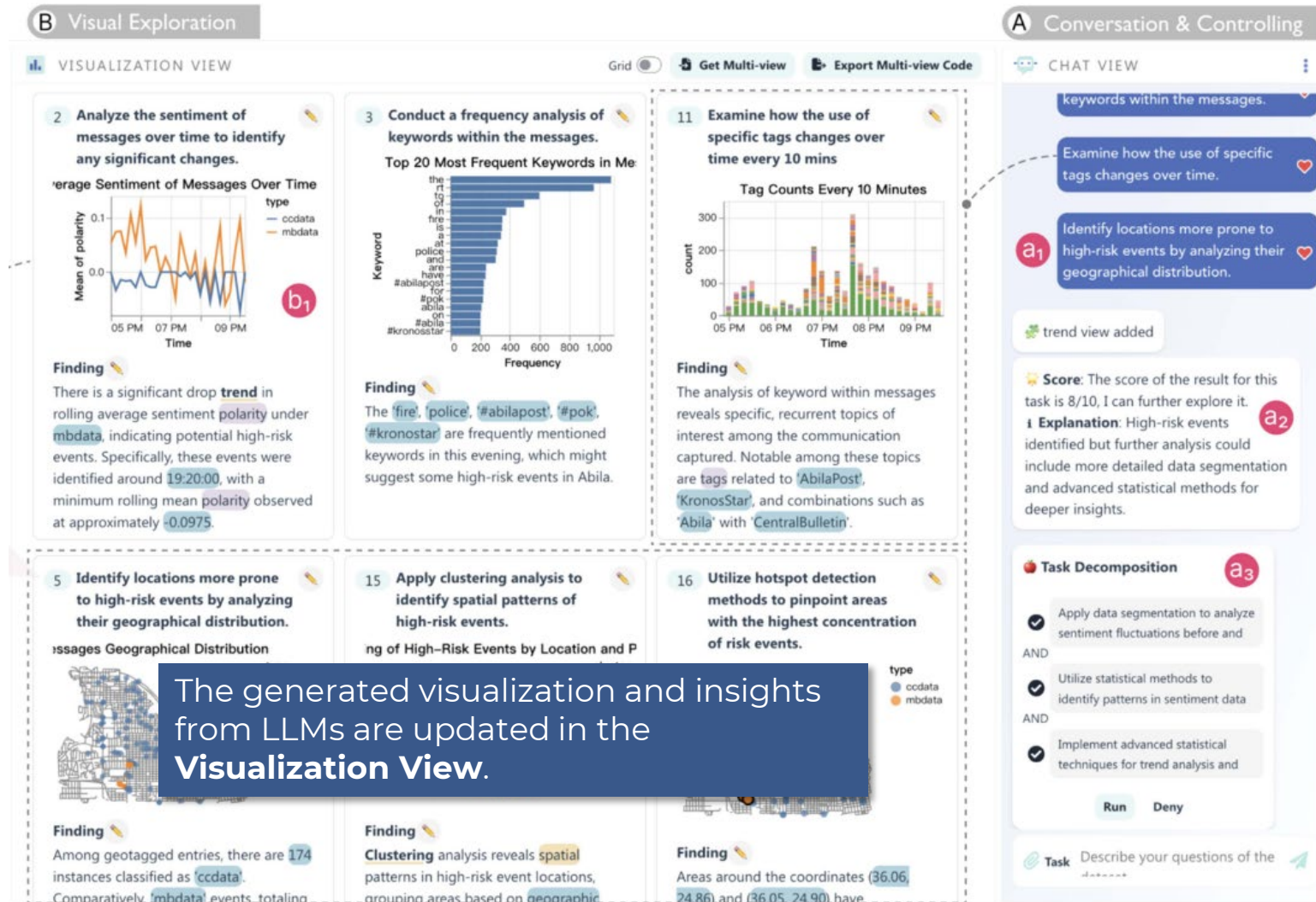


LightVA



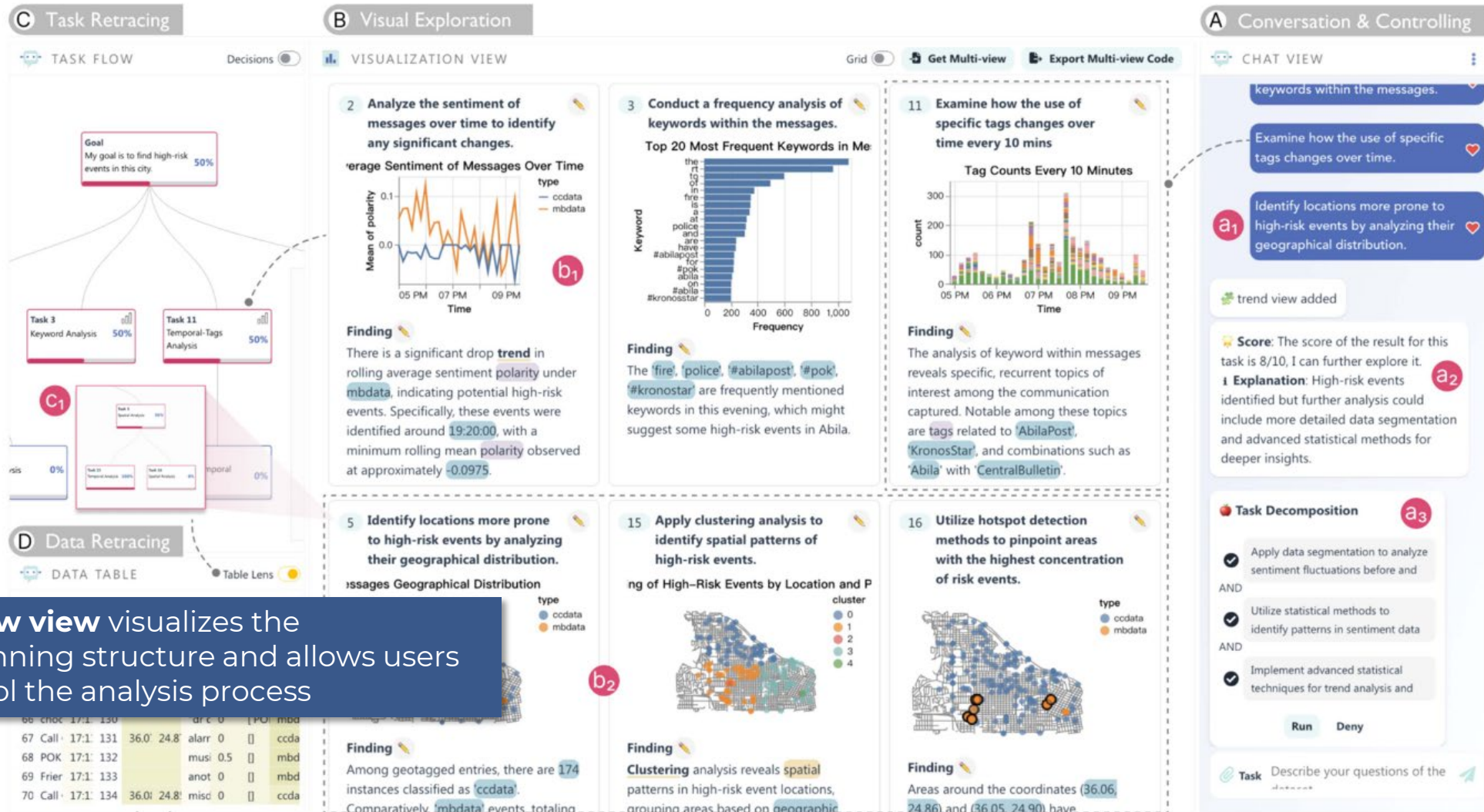
Users communicate with LLMs and control the planning process in the **Chat View** by selecting the tasks or setting the decomposition plan.

LightVA



The generated visualization and insights from LLMs are updated in the **Visualization View**.

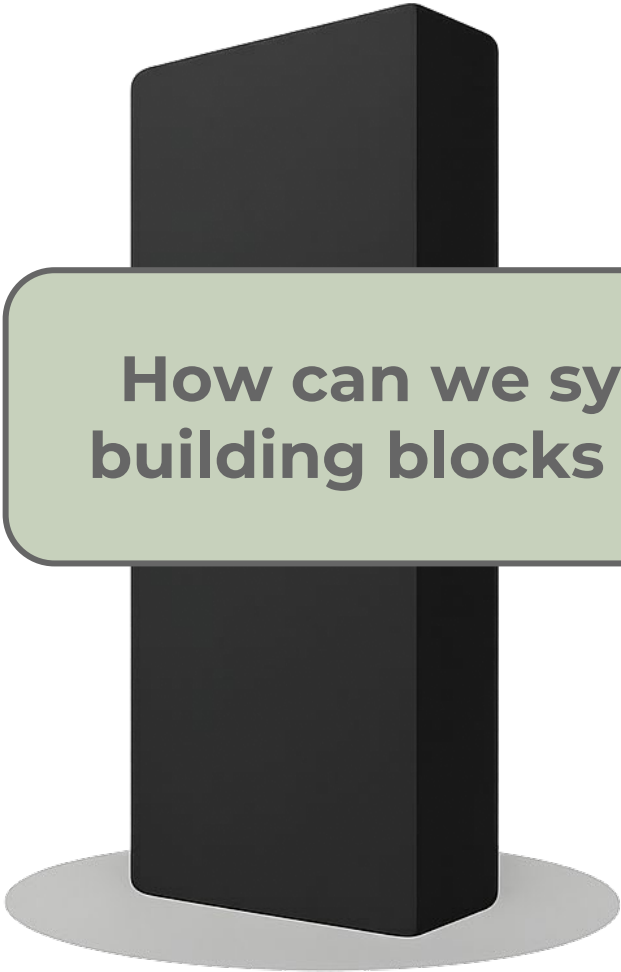
LightVA



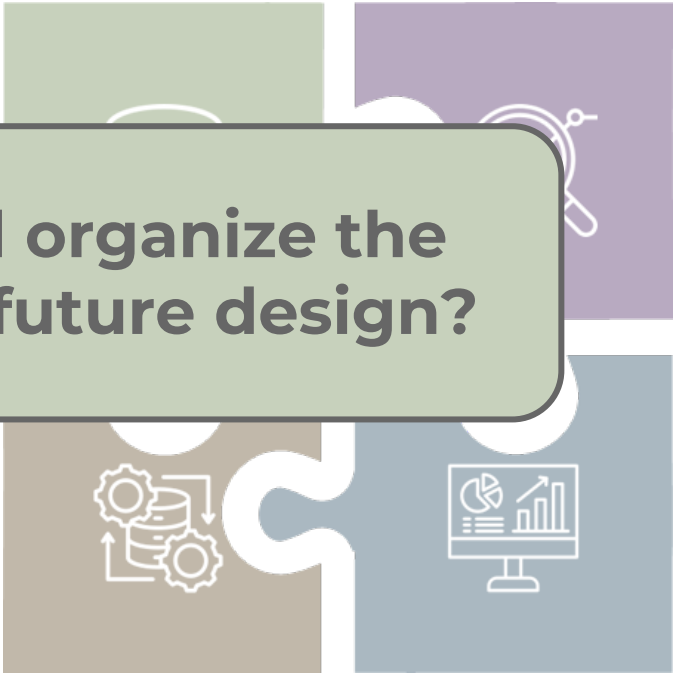
LightVA: Limitations & future work

- **LLM reliability & correctness:** planning, code generation, and insight summaries can be wrong or inconsistent; users need guardrails and verification.
- **Transparency of task planning/decomposition:** users may not understand *why* a task was suggested or decomposed; the rationale can be unclear.
- **Limited “design knowledge” for visualization quality:** chart correctness/clarity/accessibility isn’t guaranteed; the system focuses more on “getting something working” than polished design.

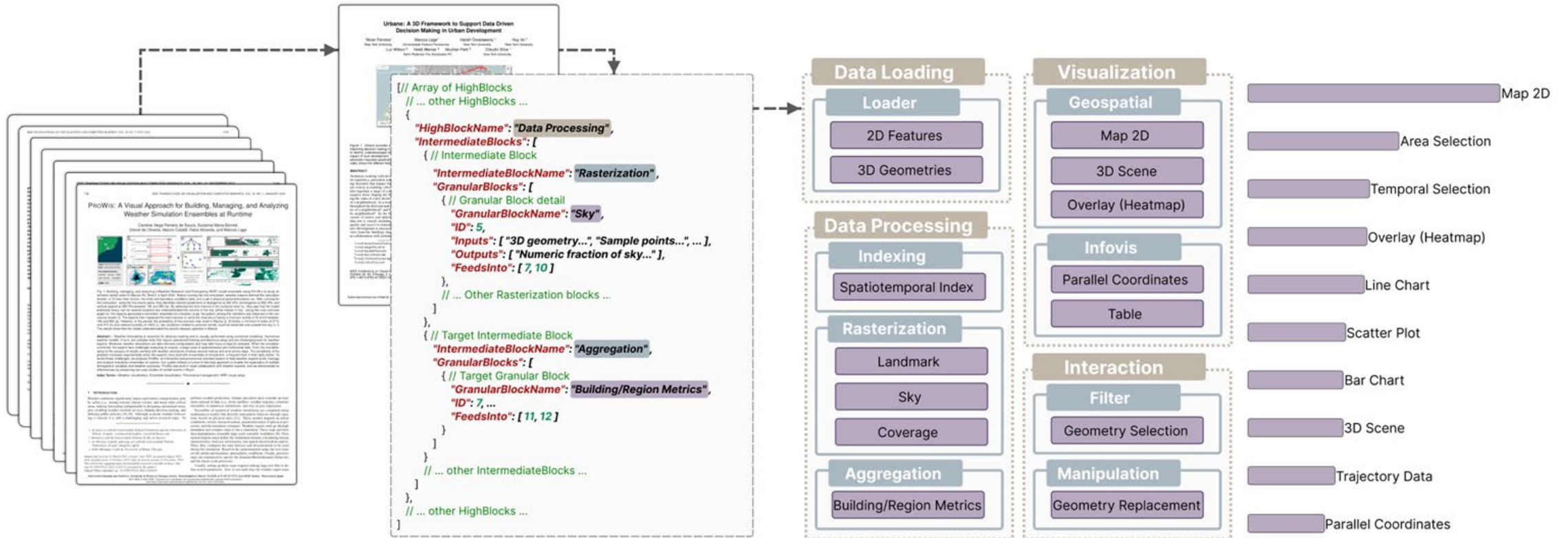
VA-Blueprint



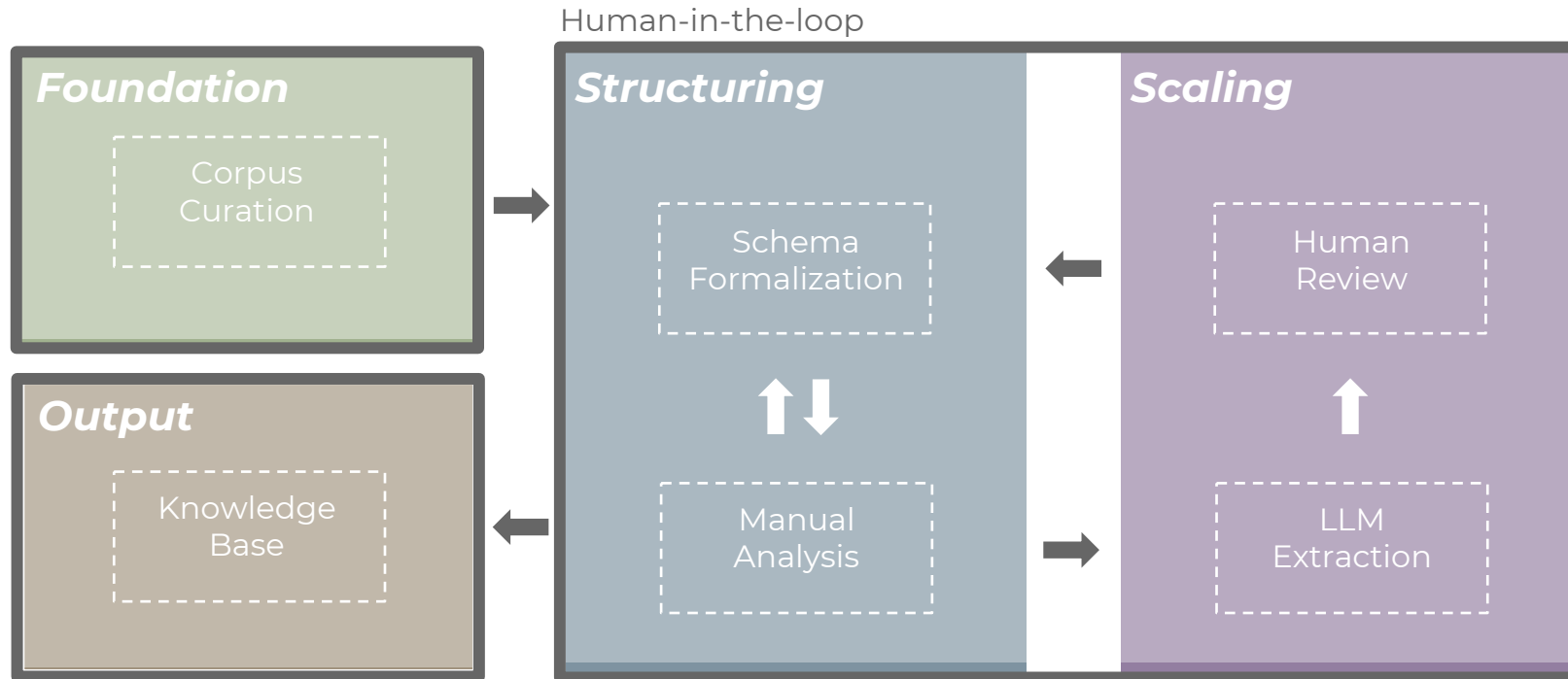
How can we systematically extract and organize the building blocks of VA systems to guide future design?



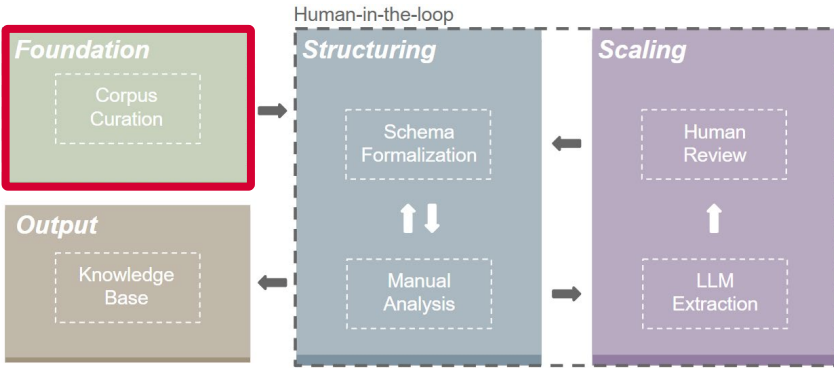
VA-Blueprint



VA-Blueprint: Overview



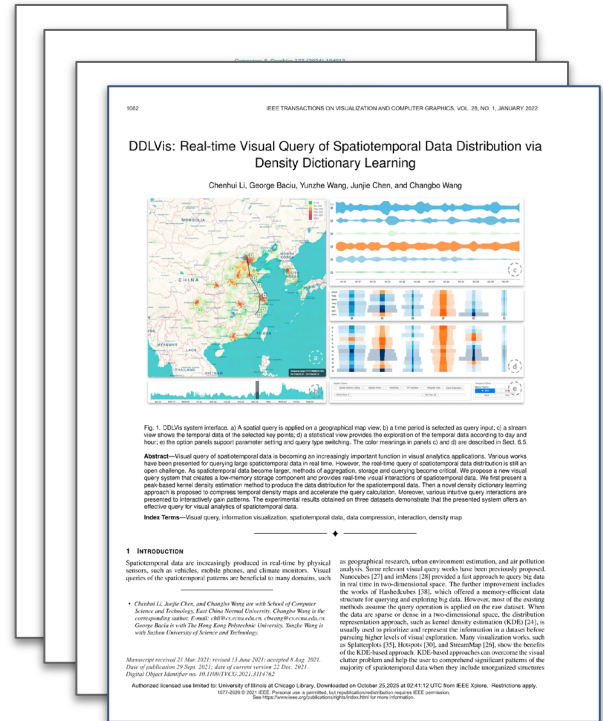
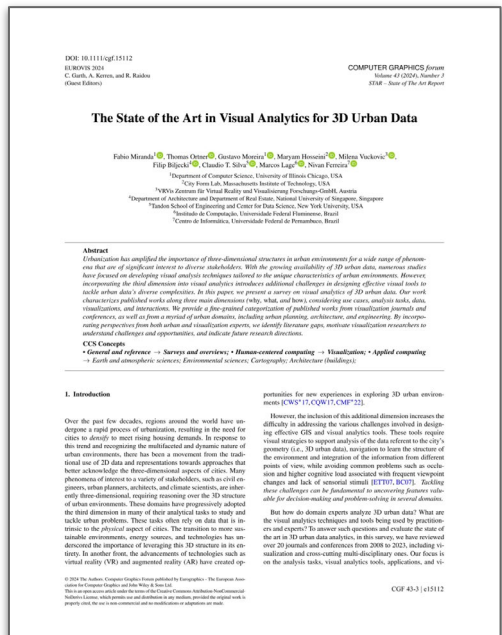
VA-Blueprint: Foundation



Ferreira et al., 2025



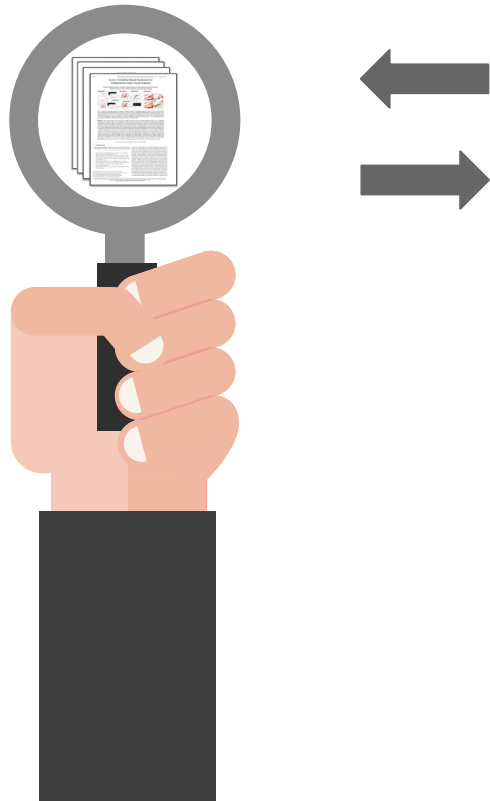
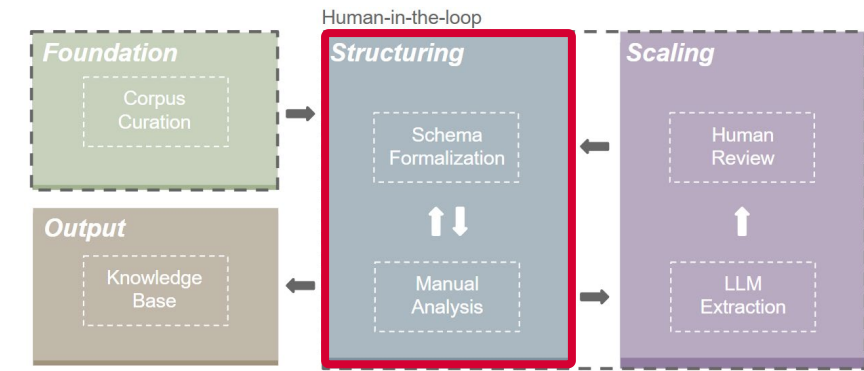
Miranda et al., 2024



85 papers

16 papers

VA-Blueprint: Structuring



VA-Blueprint Schema

```

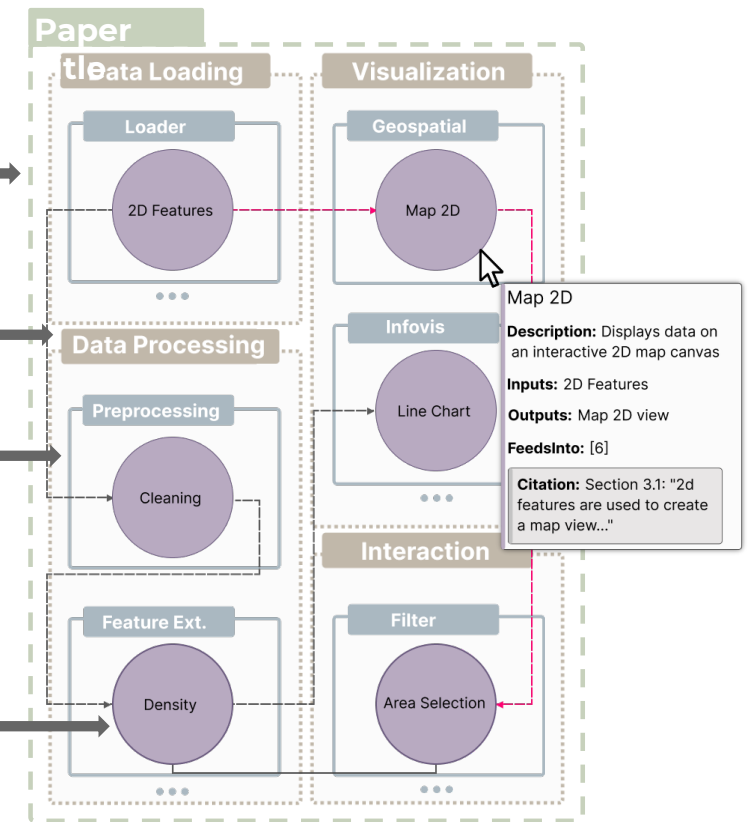
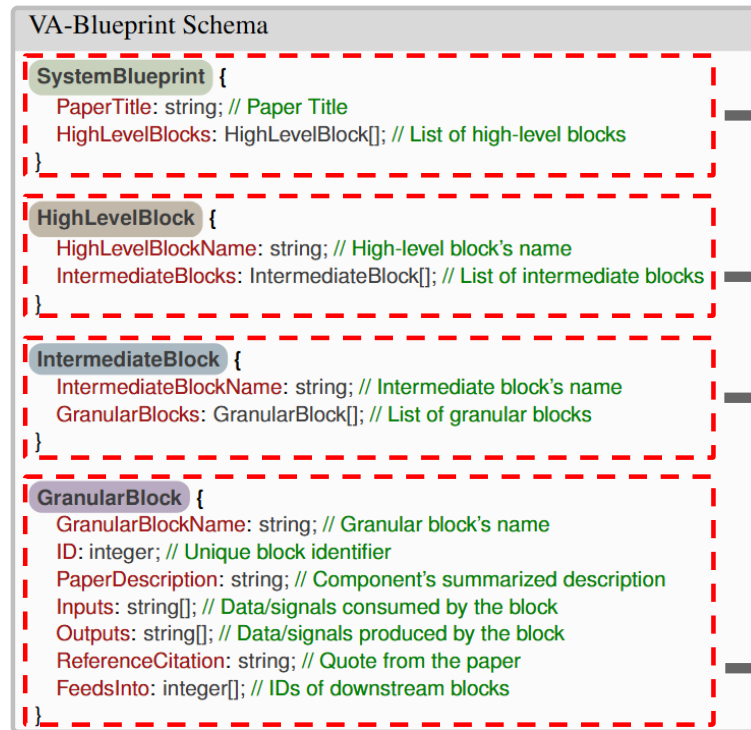
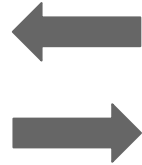
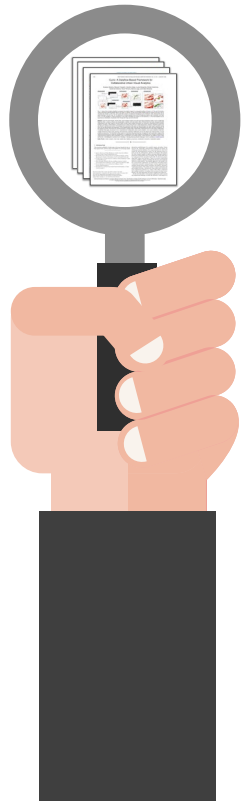
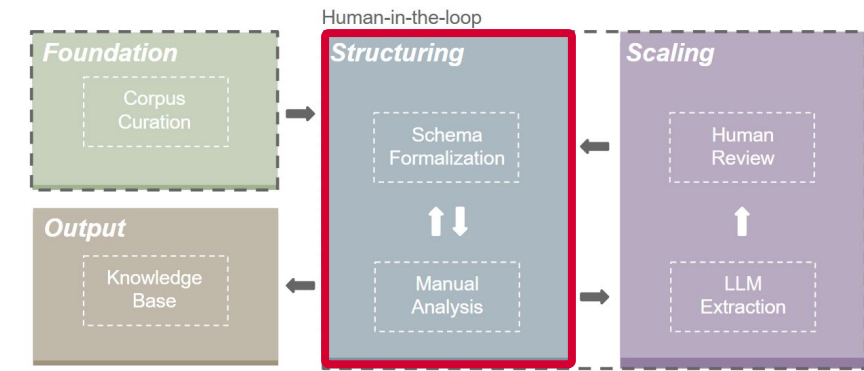
SystemBlueprint {
  PaperTitle: string; // Paper Title
  HighLevelBlocks: HighLevelBlock[]; // List of high-level blocks
}

HighLevelBlock {
  HighLevelBlockName: string; // High-level block's name
  IntermediateBlocks: IntermediateBlock[]; // List of intermediate blocks
}

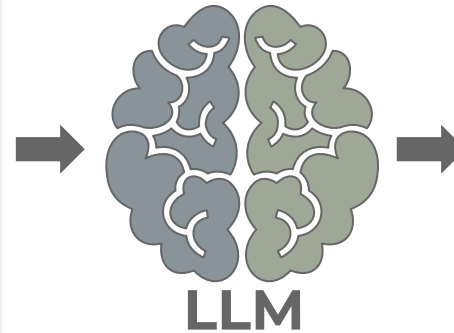
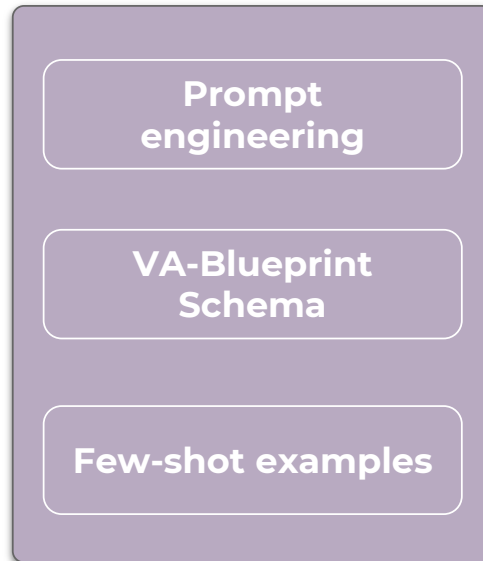
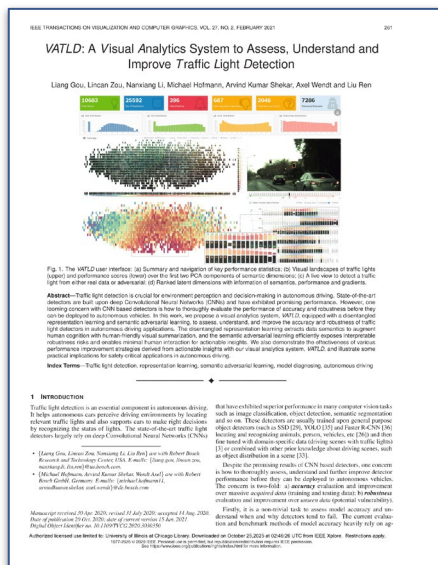
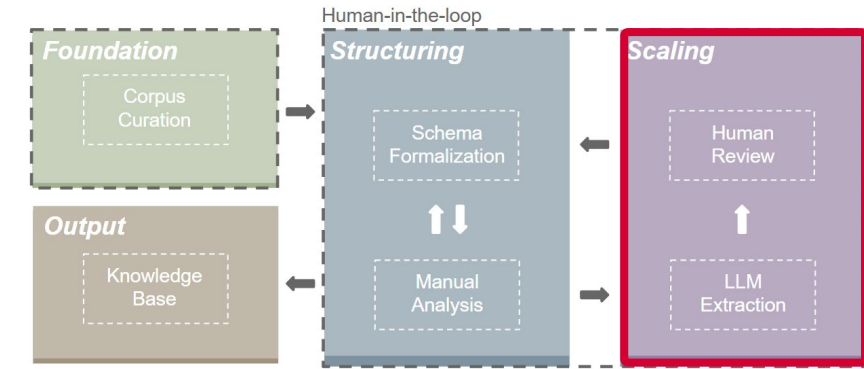
IntermediateBlock {
  IntermediateBlockName: string; // Intermediate block's name
  GranularBlocks: GranularBlock[]; // List of granular blocks
}

GranularBlock {
  GranularBlockName: string; // Granular block's name
  ID: integer; // Unique block identifier
  PaperDescription: string; // Component's summarized description
  Inputs: string[]; // Data/signals consumed by the block
  Outputs: string[]; // Data/signals produced by the block
  ReferenceCitation: string; // Quote from the paper
  FeedsInto: integer[]; // IDs of downstream blocks
}
    
```

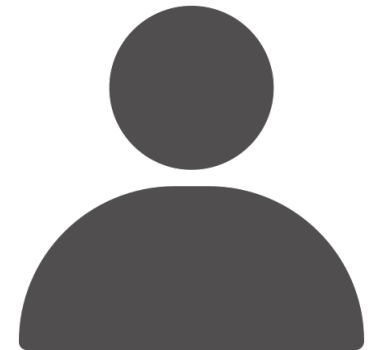
VA-Blueprint: Structuring



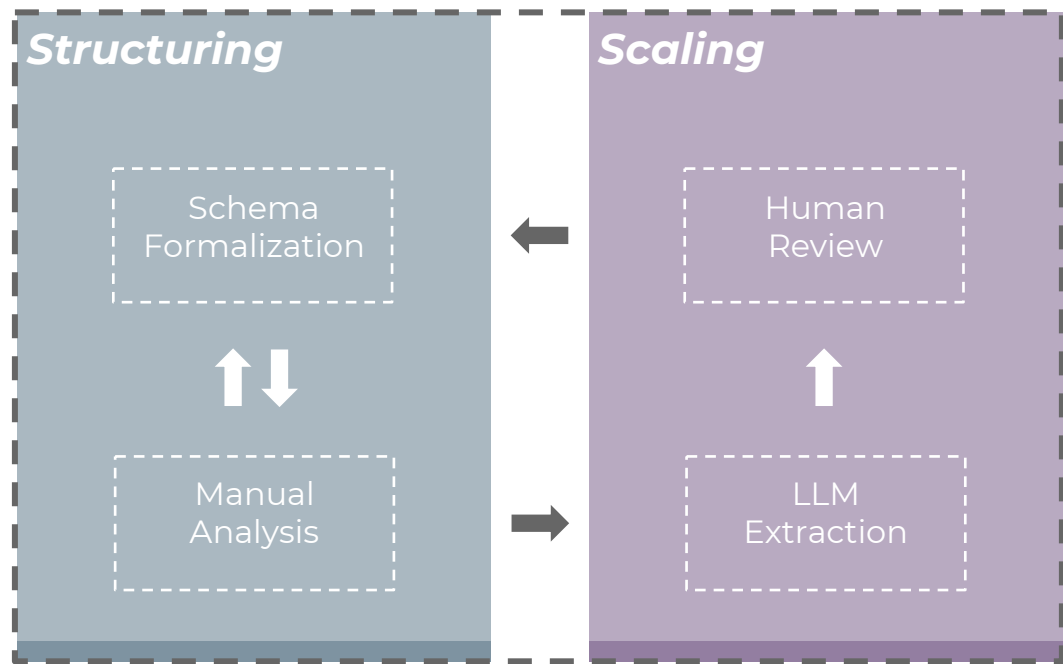
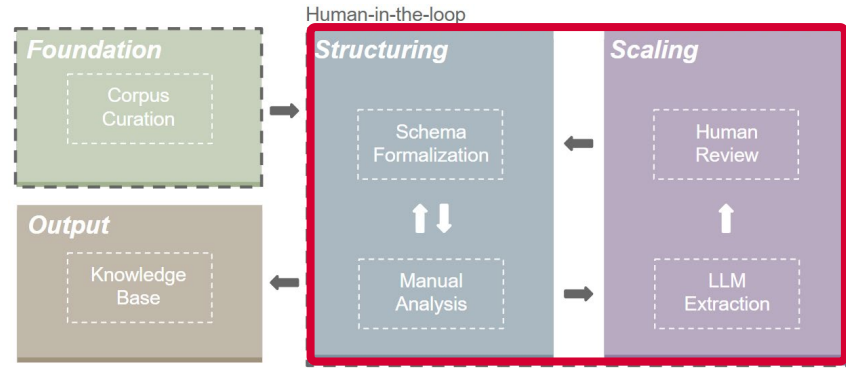
VA-Blueprint: Scaling



```
// Array of HighLevelBlocks
// ... other HighLevelBlocks ...
{
  "HighLevelBlockName": "Data Processing",
  "IntermediateBlocks": [
    // IntermediateBlock
    {
      "IntermediateBlockName": "Rasterization",
      "GranularBlocks": [
        // GranularBlock detail
        {
          "GranularBlockName": "Sky",
          "ID": 5,
          "Inputs": ["3D geometry...", "Sample points...", ...],
          "Outputs": ["Numeric fraction of sky...", ...],
          "Feedsinto": [ 7, 10 ]
        },
        // ... Other Rasterization blocks ...
      ]
    },
    // Target IntermediateBlock
    {
      "IntermediateBlockName": "Aggregation",
      "GranularBlocks": [
        // Target GranularBlock
        {
          "GranularBlockName": "Building/Region Metrics",
          "ID": 7,
          "Feedsinto": [ 11, 12 ]
        },
        // ... other IntermediateBlocks ...
      ]
    },
    // ... other HighLevelBlocks ...
  ]
}
```



VA-Blueprint: Human-in-the-loop



```
VA-Blueprint Schema

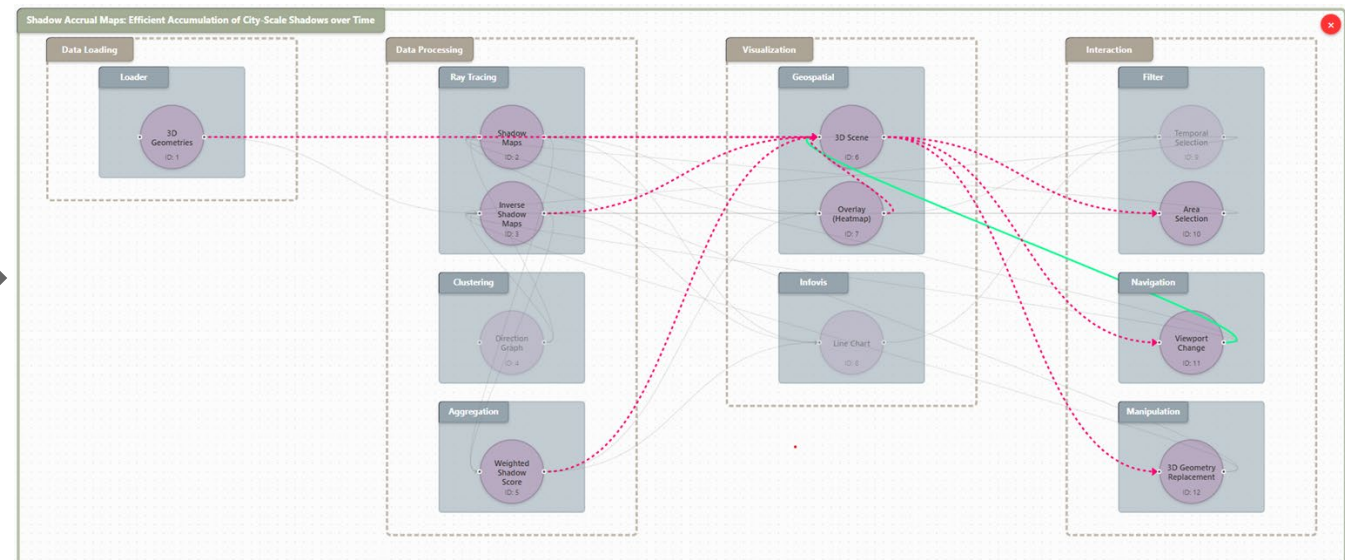
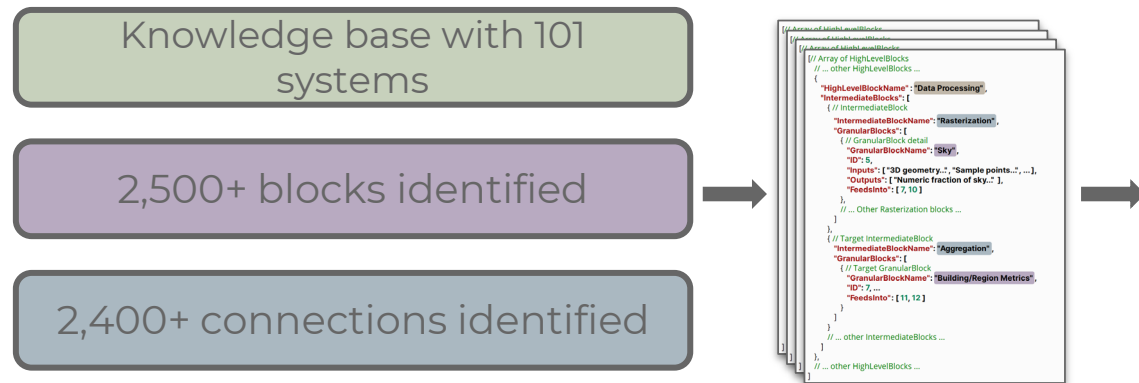
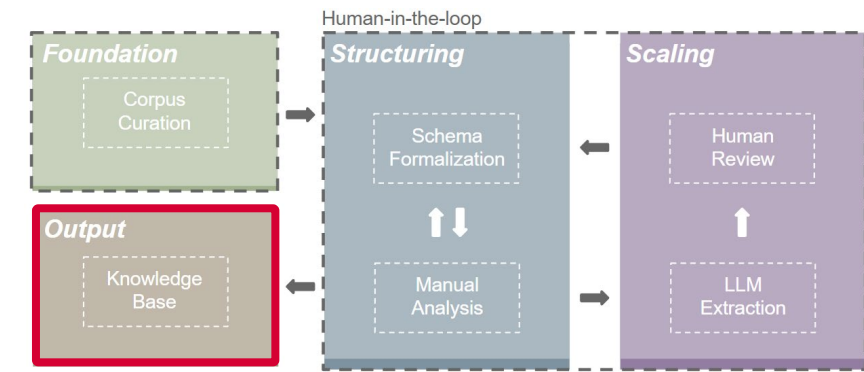
SystemBlueprint {
  PaperTitle: string; // Paper Title
  HighLevelBlocks: HighLevelBlock[]; // List of high-level blocks
}

HighLevelBlock {
  HighLevelBlockName: string; // High-level block's name
  IntermediateBlocks: IntermediateBlock[]; // List of intermediate blocks
}

IntermediateBlock {
  IntermediateBlockName: string; // Intermediate block's name
  GranularBlocks: GranularBlock[]; // List of granular blocks
}

GranularBlock {
  GranularBlockName: string; // Granular block's name
  ID: integer; // Unique block identifier
  PaperDescription: string; // Component's summarized description
  Inputs: string[]; // Data/signals consumed by the block
  Outputs: string[]; // Data/signals produced by the block
  ReferenceCitation: string; // Quote from the paper
  FeedsInto: integer[]; // IDs of downstream blocks
}
```


VA-Blueprint: Output



VA-Blueprint: Experts' feedback



accuracy and completeness

hierarchical representation effectiveness

potential usefulness

VA-Blueprint: Experts' feedback

Positive

"99% ok"

"Both represent the systems globally very well"

Hierarchical structure fits system organization

Valued for system studies and meta-analysis

Understanding system evolution through versioning

Negative

Very few missing components

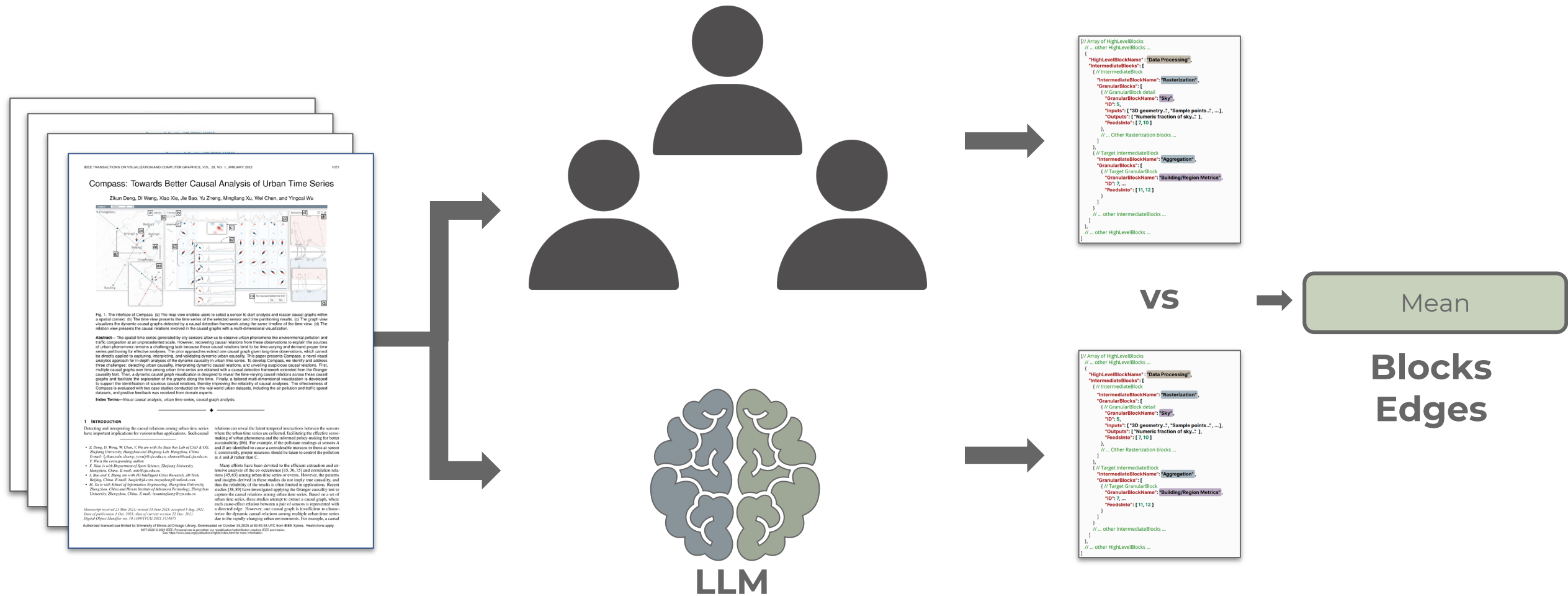
Merging different functional steps into one block

Missing edges

Apply hierarchy to connections too

Distinguish preprocessing from on-the-fly processes

VA-Blueprint: Quantitative analysis



VA-Blueprint: Quantitative analysis

METRIC	MANUAL	LLM
Edges	18.80 $\pm$ 6.77	16.65 $\pm$ 6.98
Intermediate blocks	7.65 $\pm$ 1.76	6.90 $\pm$ 1.37
Granular blocks	13.60 $\pm$ 3.28	11.30 $\pm$ 2.77

Data Processing 22% ↓

Of all granular blocks (228),
226 were correct

VA-Blueprint: Limitations & future work

LLM extraction limits

Issue: Model under-extracts or merges distinct components.

Next: Incorporate multimodal extraction, code source, and larger-context models to improve granularity.

Semantic ambiguity

Issue: Some components remain semantically ambiguous or defined at inconsistent levels of detail.

Next: Refine taxonomy and hierarchy rules to ensure clearer distinctions and consistent abstraction across components.

Domain scope

Issue: Current corpus limited to urban VA systems.

Next: Expand to other domains to test schema generalizability.